

CHƯƠNG 4: LẬP TRÌNH JSP

Mục tiêu:

Cung cấp cho sinh viên kỹ năng lập trình java web sử dụng công nghệ JSP tạo ra các trang web động tạo nội dung dựa trên các thẻ hoặc biểu thức Java. Sử dụng các đối tượng ẩn(implicit objects) như request, response, session, application.. giúp quản lý dữ liệu và trạng thái của ứng dụng. Kết hợp với Servlet đã được học ở chương 3 và ứng dụng trong mô hình MVC (Model-View-Controller) để tách biệt logic dữ liệu (Model), hiển thị (View), điều khiển (Controller), kết hợp với các tính năng bảo mật của java.

Nội dung:

- Vòng đời, cấu trúc và tiến trình của một JSP.
- Các cú pháp chính trong JSP, Các Thẻ JSP, các đối tượng ẩn(Implicit), Thư viện thẻ chuẩn Standard Tag Library (JSTL), Client Request, Server Response, JavaBeans, xử lý ngoại lệ(Exception), gỡ lỗi và bảo mật trong JSP .

Bài 7: Lập trình JSP.

JavaServer Pages (JSP) là một công nghệ phát triển web trong Java, cho phép kết hợp mã Java và HTML để tạo ra trang web động. JSP Thường được tích hợp với Servlet để cung cấp giải pháp toàn diện cho phát triển ứng dụng web trong Java. Trong nội dung bài này chúng ta sẽ tìm hiểu những khái niệm cơ bản của JSP như cấu trúc, vòng đời, các đối tượng ẩn và các điều khiển luồng trong JSP.

4.1. Java Server Pages (JSP) là gì?

JavaServer Pages (JSP) là một công nghệ để phát triển các Webpage mà hỗ trợ nội dung động, giúp các lập trình viên chèn java code vào trong các HTML page bằng việc sử dụng các JSP tag đặc biệt, hầu hết bắt đầu với <% và kết thúc với %>.

Một thành phần JavaServer Pages (JSP) là một loại Java Servlet, được thiết kế để thực hiện vai trò của một giao diện người dùng (User Interface) cho một ứng dụng Java Web. Lập trình viên Web viết JSPs như là các text file mà kết nối HTML code hoặc XHTML code, phần tử XHTML, và các JSP actions và câu lệnh JSP được nhúng.

Sử dụng JSP, bạn có thể thu thập input từ người dùng thông qua các Webpage Form, trình bày các record từ một Database hoặc đăng ký quyền ưu tiên của người dùng, truy cập các thành phần JavaBeans, truyền điều khiển giữa các page và chia sẻ thông tin giữa các Request, page, ...

4.1.1. Tại sao sử dụng JSP?

JavaServer Pages (JSP) thường hoạt động với cùng mục đích như các chương trình độc lập bởi sử dụng Common Gateway Interface (CGI). Nhưng JSP thường có một số lợi thế trong khi so sánh với CGI:

- Hiệu năng tốt hơn đáng kể bởi vì JSP cho phép nhúng các phần tử động trong chính các HTML page thay vì phải có một CGI file riêng biệt.
- JSP luôn luôn được biên dịch trước khi nó được xử lý bởi Server, không giống như CGI/Perl mà yêu cầu Server tải một trình thông dịch (Interpreter) và Target Script mỗi khi trang được yêu cầu.
- JavaServer Pages (JSP) được xây dựng ở trên cùng của Java Servlet API, vì thế, giống Servlet, JSP cũng có quyền truy cập tới tất cả Enterprise Java APIs mạnh mẽ, bao gồm JDBC, EJB, JAXP, ...
- Các JSP page có thể được sử dụng để kết nối với Servlet mà xử lý Business logic, mô hình được hỗ trợ bởi Java Servlet.
- Cuối cùng, JSP là một phần toàn bộ của Java EE, một Platform đầy đủ cho các ứng dụng lớp Enterprise. Nghĩa là, JSP có thể hoạt động như là một phần trong các ứng dụng đơn giản nhất cũng như các ứng dụng phức tạp.

Lợi thế của JSP

Dưới đây là các lợi thế của việc sử dụng JSP khi so sánh với các công nghệ khác:

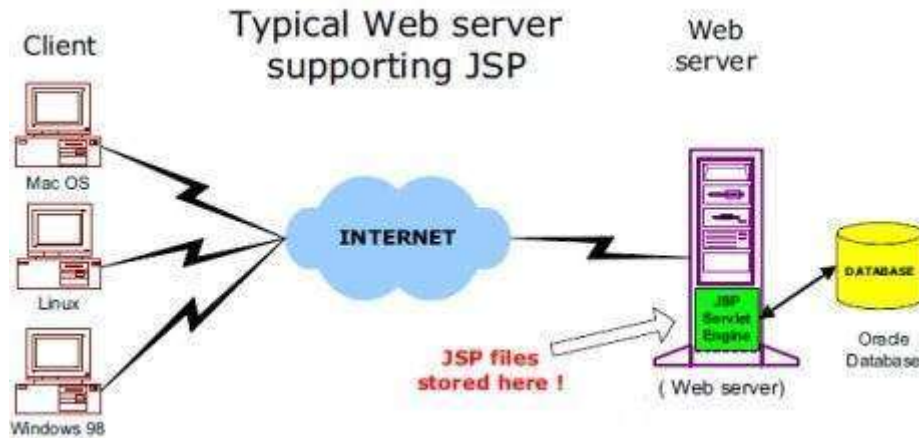
- So với Active Server Pages (ASP): Lợi thế của JSP có thể coi là gấp đôi. Đầu tiên, các phần động được viết bằng Java, không phải bằng Visual Basic hoặc ngôn ngữ MS khác, vì thế nó mạnh mẽ hơn và dễ dàng để sử dụng hơn. Thứ hai, nó thích hợp cho các Hệ điều hành khác, không chỉ là Microsoft Web Server.
- So với Pure Servlets: Nó tiện lợi hơn khi viết (và sửa đổi) HTML, vì có nhiều lệnh `println` hơn.
- So với Server-Side Includes (SSI): SSI chỉ dành cho các thẻ đơn giản, không dành cho các chương trình "thực" mà sử dụng Form Data, tạo kết nối Database.
- So với JavaScript: JavaScript có thể tạo HTML động trên Client nhưng lại tương tác khó khăn với Web Server để thực hiện các tác vụ phức tạp như truy cập Database và xử lý hình ảnh, ...
- Với Static HTML: Tất nhiên, HTML thông thường không thể chứa thông tin động.

4.1.2. Cấu trúc JSP

Web Server cần một JSP engine (ví dụ: Container) để xử lý các JSP page. JSP container có nhiệm vụ chặn các Request cho các JSP page. Bài hướng dẫn này sử dụng Apache, mà có JSP container có sẵn, để hỗ trợ phát triển các JSP page.

Một JSP container làm việc với Web Server để cung cấp môi trường runtime và các dịch vụ khác mà một JSP cần đến. Nó biết cách để hiểu các phần tử đặc biệt mà là một phần của JSP.

Sơ đồ sau minh họa vị trí của JSP container và JSP file trong một Web Application.

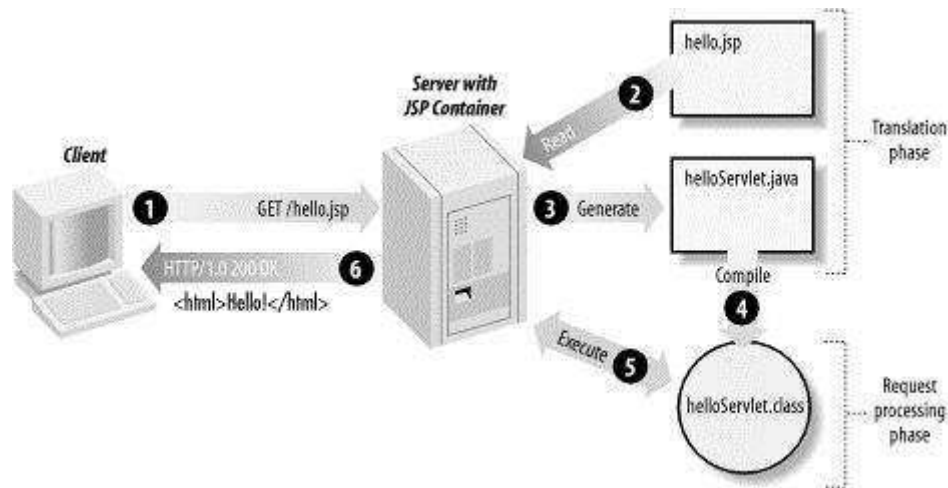


Hình 4: 1 JSP Container

4.1.4. Tiến trình JSP

Dưới đây là các bước giải thích cách Web Server tạo các Webpage bởi sử dụng JSP:

- Với một trang thông thường, trình duyệt gửi một HTTP request tới Web server.
- Web server nhận ra rằng HTTP request là cho một JSP page và chuyển nó tới một JSP engine. Điều này được thực hiện bởi sử dụng một URL hoặc một JSP page mà kết thúc với .jsp thay vì .html.
- JSP engine tải JSP page từ đĩa và biến đổi nó thành một nội dung Servlet. Sự biến đổi này là rất đơn giản, trong đó tất cả Template Text được biến đổi thành các lệnh `println()` và tất cả phần tử JSP được biến đổi thành Java code mà triển khai các hành vi động tương ứng của trang.
- JSP engine biên dịch Servlet thành một lớp có thể thực thi và chuyển Request ban đầu tới một Servlet engine.
- Một phần của Web server triệu hồi Servlet engine tải lớp Servlet và thực thi nó. Trong quá trình thực thi, Servlet này tạo một output trong định dạng HTML, mà Servlet engine truyền tới Web server bên trong một HTTP response.
- Web server chuyển HTTP Response tới trình duyệt của bạn nội dung HTML tĩnh.
- Cuối cùng, trình duyệt web xử lý trang HTML đã được tạo một cách năng động bên trong HTTP response một cách chính xác như kiểu nó là một trang tĩnh.
- Tất cả các bước trên có thể được minh họa trong sơ đồ sau:



Hình 4: 2 Tiến trình JSP

Nét đặc trưng là, JSP engine kiểm tra để biết có hay không một Servlet cho một JSP file là đã tồn tại và có hay không ngày sửa đổi JSP là cũ hơn Servlet đó. Nếu JSP là cũ hơn Servlet đã tạo của nó, thì JSP engine giả sử rằng JSP đã không thay đổi và Servlet đã được tạo đó vẫn kết nối với nội dung của JSP. Điều này làm cho tiến trình khác biệt hơn khi so với các ngôn ngữ Scripting khác (ví dụ như PHP) và vì thế nó nhanh hơn.

Trước khi hiển thị ra trình duyệt, tập tin JSP phải được biên dịch thành Servlet, dùng bộ biên dịch JSP (JSP compiler). Bộ biên dịch JSP có thể tạo servlet thành mã nguồn Java trước, rồi biên dịch mã nguồn ra tập tin .class dùng bộ biên dịch Java, hoặc có thể trực tiếp tạo mã byte code cho servlet từ trang JSP. Ngoài trừ giai đoạn biên dịch, một JSP page được xử lý chính xác giống như một Servlet thông thường.

4.2. Vòng đời (Life Cycle) trong JSP

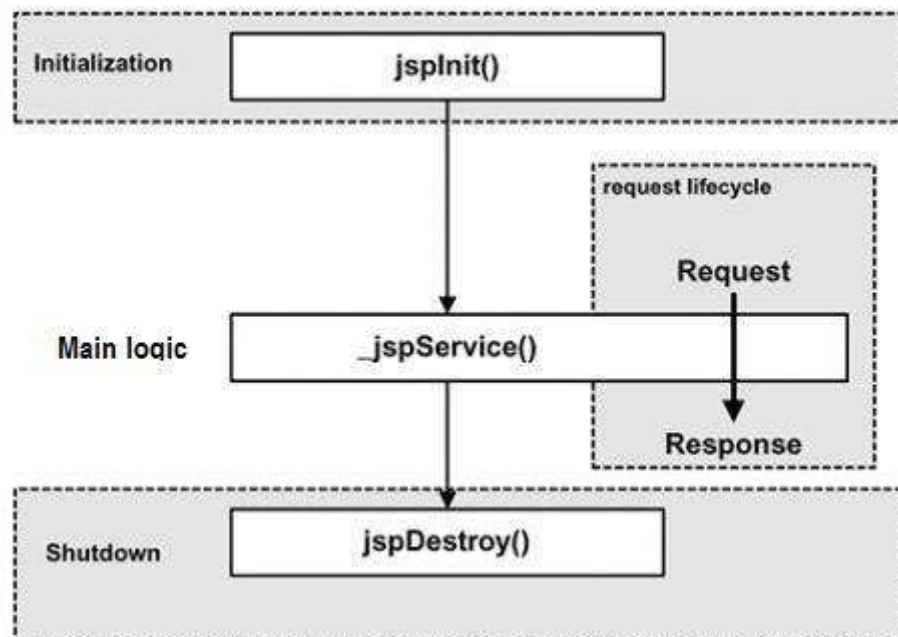
Chìa khóa để tìm hiểu tính năng tầm thấp của JSP là hiểu vòng đời của nó.

Vòng đời của JSP có thể được định nghĩa như là toàn bộ tiến trình từ khi tạo ra đến khi hủy nó, tương tự như vòng đời của một Servlet, nhưng thêm một bước để biên dịch một JSP thành Servlet.

Sau đây là các giai đoạn trong vòng đời của của một JSP:

1. Biên dịch
2. Khởi tạo
3. Thực thi
4. Hủy

Bốn giai đoạn chủ yếu trong vòng đời của JSP là tương tự như vòng đời của Servlet và chúng như sau:



Hình 4: 3 Vòng đời của JSP

Biên dịch JSP

Khi một trình duyệt yêu cầu một JSP, đầu tiên JSP engine kiểm tra để biết có hay không nó cần biên dịch trang này. Nếu trang chưa được biên dịch, hoặc nếu JSP đã được sửa đổi từ lần sửa đổi cuối cùng, thì JSP engine biên dịch trang này.

Tiến trình biên dịch gồm 3 bước:

- 1) Parse một JSP
- 2) Biên dịch JSP thành một Servlet
- 3) Biên dịch Servlet đó.

Khởi tạo JSP

Khi một container tải một JSP, nó gọi phương thức `jspInit()` trước khi phục vụ bất kỳ yêu cầu nào. Nếu bạn cần thực hiện sự khởi tạo JSP riêng, ghi đề phương thức `jspInit()`:

```

public void jspInit(){
    // Initialization code...
}
  
```

Nét đặc trưng là, việc khởi tạo được thực hiện chỉ một lần và với phương thức `init` của Servlet, nói chung bạn khởi tạo các kết nối Database, mở các file, và tạo các bảng tra cứu trong phương thức `jspInit`.

Thực thi JSP

Giai đoạn này trong vòng đời JSP biểu diễn tất cả các tương tác với Request tới khi JSP đó bị hủy.

Bất cứ khi nào một trình duyệt yêu cầu một JSP và trang đã được tải và được khởi tạo, thì JSP engine triệu hồi phương thức `_jspService()` trong JSP đó.

Phương thức `_jspService()` nhận một `HttpServletRequest` và một `HttpServletResponse` như là các tham số của nó.

```
void _jspService(HttpServletRequest request,
                  HttpServletResponse response)
{
    // Service handling code...
}
```

Phương thức `_jspService()` của một JSP được triệu hồi một lần cho mỗi yêu cầu và nó chịu trách nhiệm tạo Response cho Request đó và phương thức này cũng chịu trách nhiệm tạo các phản hồi tới tất cả 7 phương thức của HTTP, ví dụ: GET, POST, DELETE.

Hủy JSP

Giai đoạn hủy một JSP trong vòng đời JSP biểu thị khi nào thì một JSP bị gỡ bỏ khỏi một container.

Phương thức `jspDestroy()` trong JSP là phương thức hủy tương đương với trong Servlet. Ghi đề phương thức `jspDestroy` khi bạn cần thực hiện bất kỳ quá trình hủy nào, ví dụ như giải phóng kết nối với Database, hoặc đóng các file.

Phương thức `jspDestroy()` có form sau:

```
public void jspDestroy()
{
    // Your cleanup code goes here.
}
```

4.3. Cú pháp trong JSP

Phần này trình bày về cú pháp đơn giản (ví dụ: các phân tử) liên quan tới lập trình JSP.

4.3.1. Scriptlet trong JSP

Scriptlet có thể chứa bất kỳ số lượng lệnh, biến, khai báo phương thức hoặc Expression của ngôn ngữ Java nào mà là hợp lệ trong ngôn ngữ Scripting.

Sau đây là cú pháp của Scriptlet trong JSP:

```
<% code fragment %>
```

Có thể viết XML tương đương với cú pháp trên như sau:

```
<jsp:scriptlet>
```

```
code fragment
</jsp:scriptlet>
```

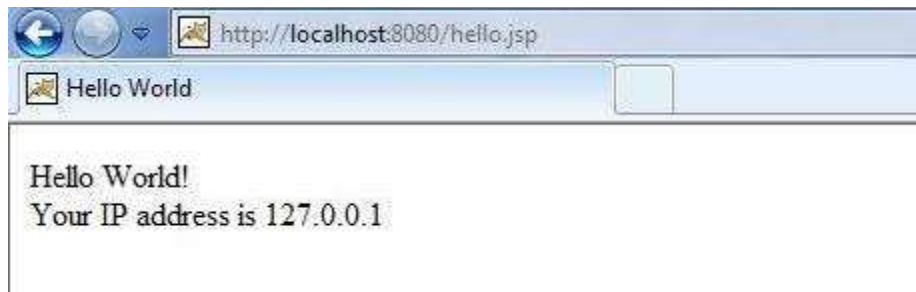
Bất kỳ text, thẻ HTML, hoặc phần tử JSP nào bạn viết phải ở bên ngoài Scriptlet đó. Sau đây là ví dụ đơn giản cho JSP:

```
<html>
<head><title>Hello World</title></head>
<body>
Hello World!<br/>
<%
out.println("Your IP address is " + request.getRemoteAddr());
%>
</body>
</html>
```

Chú ý: Giả sử rằng Apache Tomcat được cài đặt trong C:\apache-tomcat-7.0.2 và môi trường của bạn được cài đặt theo hướng dẫn.

<http://localhost:8080/hello.jsp>. Nó sẽ cho kết quả sau:

Ví dụ JSP đầu tiên



Hình 4: 4 Kết quả chương trình Hello World JSP

4.3.2. Declarations trong JSP

Một Declaration khai báo một hoặc nhiều biến hoặc phương thức mà có thể sử dụng trong Java code và sau đó trong Java file. Bạn phải khai báo biến hoặc phương thức trước khi bạn sử dụng nó trong JSP file.

Sau đây là cú pháp của JSP Declaration:

```
<%! declaration; [ declaration; ]+ ... %>
```

Bạn có thể viết XML tương đương với cú pháp trên như sau:

```
<jsp:declaration>
code fragment
```

```
</jsp:declaration>
```

Dưới đây là ví dụ đơn giản cho JSP Declaration:

```
<%! int i = 0; %>
<%! int a, b, c; %>
<%! Circle a = new Circle(2.0); %>
```

4.3.3. Expression trong JSP

Một phần tử Expression trong JSP chứa một biểu thức ngôn ngữ Scripting mà được tính toán, được biến đổi thành một String, và được chèn tại nơi Expression đó xuất hiện trong JSP file.

Bởi vì giá trị của một Expression được biến đổi thành một String, bạn có thể sử dụng một Expression bên trong một dòng text, có hoặc không nó được tag với HTML, trong một JSP file.

Phần tử Expression có thể chứa bất kỳ expression nào mà là hợp lệ với Java Language Specification, nhưng bạn không thể sử dụng một dấu chấm phẩy ở phần cuối của một expression.

Sau đây là cú pháp cho Expression trong JSP:

```
<%= expression %>
```

Bạn có thể viết XML tương đương với cú pháp trên như sau:

```
<jsp:expression>
  expression
</jsp:expression>
```

Dưới đây là ví dụ đơn giản về Expression trong JSP:

```
<html>
<head><title>A Comment Test</title></head>
<body>
<p>
  Today's date: <%= (new java.util.Date()).toLocaleString()%>
</p>
</body>
</html>
```

Nó sẽ cho kết quả sau:

```
Today's date: 11-Sep-2010 21:24:25
```

4.3.4. Comment trong JSP

Comment trong JSP đánh dấu text hoặc các lệnh mà JSP container nên bỏ qua. Một JSP comment là hữu ích khi bạn muốn ẩn hoặc bỏ qua phần đó trong JSP page.

Sau đây là cú pháp của Comment trong JSP:

```
<!-- This is JSP comment -->
```

Ví dụ đơn giản sau minh họa Comment trong JSP:

```
<html>
<head><title>A Comment Test</title></head>
<body>
<h2>A Test of Comments</h2>
<!-- This comment will not be visible in the page source -->
</body>
</html>
```

Nó sẽ cho kết quả sau:

A Test of Comments

Có một số chỉ lệnh đặc biệt mà ta có thể sử dụng trong nhiều trường hợp khác nhau để chèn các comment hoặc các ký tự mà được đối xử một cách đặc biệt. Sau đây là liệt kê các chỉ lệnh này:

Cú pháp	Mục đích
<%-- comment --%>	Một JSP comment. Được bỏ qua bởi JSP engine
<!-- comment -->	Một HTML comment. Được bỏ qua bởi trình duyệt
<%	Biểu diễn một static <% literal.
%>	Biểu diễn một static %> literal.
\'	Một trích dẫn đơn trong một thuộc tính mà sử dụng các trích dẫn đơn
\"	Một trích dẫn kép trong một thuộc tính mà sử dụng các trích dẫn kép

Bảng 4: 1 Comment trong JSP

4.3.5. Directive trong JSP

Một directive trong JSP ảnh hưởng tới cấu trúc tổng thể của lớp Servlet. Nó thường có form sau:

```
<%@ directive attribute="value" %>
```

Có 3 loại thẻ directive trong JSP:

Directive	Miêu tả
<%@ page ... %>	Định nghĩa một thuộc tính page-dependent (phụ thuộc trang), như ngôn ngữ scripting, trang lỗi và các yêu cầu bộ đệm
<%@ include ... %>	Include một file trong suốt giai đoạn biên dịch
<%@ taglib ... %>	Khai báo một thư viện thẻ, chứa các action tùy biến, được sử dụng trong trang đó

Bảng 4: 2 Thẻ directive trong JSP

Ta giải thích JSP Directive trong chương riêng trong phần: Directive trong JSP

4.3.6. Action trong JSP

Action trong JSP sử dụng các chỉ lệnh trong cú pháp XML để điều khiển hành vi của Servlet engine. Bạn có thể chèn động một file, tái sử dụng các thành phần JavaBeans, chuyển người sử dụng tới trang khác, hoặc tạo HTML cho Java Plugin.

Chỉ có một cú pháp cho phần tử action trong JSP, khi nó phù hợp tiêu chuẩn XML:

```
<jsp:action_name attribute="value" />
```

Về cơ bản, các phần tử action trong JSP là các hàm được định nghĩa trước và có các JSP action sau:

Cú pháp	Mục đích
jsp:include	Include một file tại thời điểm trang đó được yêu cầu
jsp:useBean	Tìm kiếm hoặc thuyết minh một JavaBean

jsp:setProperty	Thiết lập thuộc tính của một JavaBean
jsp:getProperty	Chèn thuộc tính của một JavaBean vào trong output
jsp:forward	Chuyển người yêu cầu tới trang mới
jsp:plugin	Tạo code cho trình duyệt riêng mà tạo một thẻ OBJECT hoặc EMBED cho Java plugin
jsp:element	Định nghĩa động các phần tử XML
jsp:attribute	Định nghĩa động thuộc tính của phần tử XML đã được định nghĩa
jsp:body	Định nghĩa động phần thân của phần tử XML đã được định nghĩa
jsp:text	Sử dụng để viết template text trong các JSP page và tài liệu JSP

Bảng 4: 3 Các JSP action

Chúng ta sẽ tìm hiểu JSP action trong phần riêng Action trong JSP

4.3.7. Các đối tượng ẩn (Implicit) trong JSP

JSP hỗ trợ 9 biến được định nghĩa tự động, mà cũng được gọi là các đối tượng ẩn (implicit object). Các biến đó là:

Đối tượng	Miêu tả
request	Đây là đối tượng HttpServletRequest mà liên kết với Request
response	Đây là đối tượng HttpServletResponse mà liên kết với Response tới Client
out	Đây là đối tượng PrintWriter được sử dụng để gửi output tới Client

session	Đây là đối tượng HttpSession mà liên kết với Request
application	Đây là đối tượng ServletContext mà liên kết với application context
config	Đây là đối tượng ServletConfig mà liên kết với page
pageContext	Sự gói gọn này sử dụng các đặc trưng Server-Specific giống như hiệu năng cao JspWriters .
page	Được sử dụng để gọi các phương thức được định nghĩa bởi lớp Servlet đã được biên dịch
Exception	Đối tượng Exception cho phép dữ liệu exception để được truy cập bởi JSP đã chỉ rõ

Bảng 4: 4 Các đối tượng ẩn trong JSP

4.4. Điều khiển luồng trong JSP

JSP cung cấp đầy đủ tính năng của Java để được nhúng trong các Web application. Bạn có thể sử dụng tất cả APIs và xây dựng tất cả các khối Java trong chương trình JSP của bạn, bao gồm các lệnh điều khiển luồng, vòng lặp, ...

Các lệnh điều khiển luồng trong JSP

Các khối if...else khởi đầu giống như một Scriptlet thông thường, nhưng Scriptlet được đóng tại mỗi dòng với HTML text được bao giữa các thẻ Scriptlet.

```

<%! int day = 3; %>
<html>
<head><title>IF...ELSE Example</title></head>
<body>
<% if (day == 1 | day == 7) { %>
    <p> Today is weekend</p>
<% } else { %>
    <p> Today is not weekend</p>
<% } %>
</body>
</html>

```

Nó sẽ cho kết quả sau:

Today is not weekend

Bây giờ, bạn xem xét khối switch...case mà có một chút khác biệt bởi sử dụng lệnh out.println() và bên trong các Scriptlet:

```
<%! int day = 3; %>
<html>
<head><title>SWITCH...CASE Example</title></head>
<body>
<%
switch(day) {
case 0:
    out.println("It\'s Sunday.");
    break;
case 1:
    out.println("It\'s Monday.");
    break;
case 2:
    out.println("It\'s Tuesday.");
    break;
case 3:
    out.println("It\'s Wednesday.");
    break;
case 4:
    out.println("It\'s Thursday.");
    break;
case 5:
    out.println("It\'s Friday.");
    break;
default:
    out.println("It\'s Saturday.");
}
%>
</body>
```

```
</html>
```

Nó sẽ cho kết quả sau:

```
It 's Wednesday.
```

Các lệnh vòng lặp trong JSP

Bạn có thể sử dụng 3 kiểu khối vòng lặp cơ bản trong Java là: for, while, và do...while trong chương trình JSP.

Dưới đây là ví dụ về vòng lặp for trong JSP:

```
<%! int fontSize; %>
```

```
<html>
<head><title>FOR LOOP Example</title></head>
<body>
<%for ( fontSize = 1; fontSize <= 3; fontSize++){ %>
    <font color="green" size="<%= fontSize %>">
        JSP Tutorial
    </font><br />
<%}%>
</body>
</html>
```

Nó sẽ cho kết quả sau:

```
JSP Tutorial
```

```
JSP Tutorial
```

```
JSP Tutorial
```

Ví dụ trên có thể được viết bởi sử dụng vòng lặp while trong JSP như sau:

```
<%! int fontSize; %>
<html>
<head><title>WHILE LOOP Example</title></head>
<body>
```

```

<%while ( fontSize <= 3){ %>
    <font color="green" size="<%= fontSize %>">
        JSP Tutorial
    </font><br />
    <%fontSize++;%>
    <%}%>
</body>
</html>

```

Nó sẽ cho kết quả sau:

```

JSP Tutorial

JSP Tutorial

JSP Tutorial

```

Các toán tử trong JSP

JSP hỗ trợ tất cả toán tử logic và toán tử số học được hỗ trợ bởi Java. Bảng sau cung cấp danh sách tất cả toán tử với quyền ưu tiên cao nhất được liệt kê ở trên cùng, và quyền thấp nhất là ở dưới cùng.

Bên trong một expression, toán tử có quyền ưu tiên cao hơn sẽ được tính toán trước.

Loại	Toán tử	Thứ tự ưu tiên
Postfix	() [] . (toán tử dot)	Trái sang phải
Unary	++ -- ! ~	Phải sang trái
Tính nhân	* / %	Trái sang phải
Tính cộng	+ -	Trái sang phải
Dịch chuyển	>> >>> <<	Trái sang phải
Quan hệ	> >= < <=	Trái sang phải

Cân bằng	== !=	Trái sang phải
Phép AND bit	&	Trái sang phải
Phép XOR bit	^	Trái sang phải
Phép OR bit		Trái sang phải
Phép AND logic	&&	Trái sang phải
Phép OR logic		Trái sang phải
Điều kiện	?:	Phải sang trái
Gán	= += -= *= /= %= >>= <<= &= ^= =	Phải sang trái
Dấu phẩy	,	Trái sang phải

Bảng 4: 5 Các toán tử trong JSP

Literal trong JSP

JSP Expression Language (EL) định nghĩa các literal sau:

- **Boolean:** true và false
- **Integer:** như trong Java
- **Floating point:** như trong Java
- **String:** với các trích dẫn đơn và kép; " được thay thế bởi \", ' được thay thế bởi \', và \ được thay thế bởi \\\.
- **Null:** null

4.5. Directive trong JSP

JSP directive cung cấp các chỉ dẫn và chỉ lệnh tới container, nói cho nó cách để xử lý các mặt cụ thể của tiến trình xử lý JSP.

Một Directive trong JSP ảnh hưởng tới cấu trúc tổng thể của lớp Servlet. Nó thường có form sau:

```
<%@ directive attribute="value" %>
```

Directive có thể có các thuộc tính mà bạn có thể liệt kê dưới dạng các cặp key-value và được phân biệt bởi dấu phẩy.

Các khoảng trống giữa ký hiệu @ và tên directive, và giữa thuộc tính cuối cùng và ký hiệu đóng %>, là tùy ý.

Có 3 loại thẻ directive trong JSP:

Directive	Miêu tả
<%@ page ... %>	Định nghĩa một thuộc tính page-dependent (phụ thuộc trang), như ngôn ngữ scripting, trang lỗi và các yêu cầu bộ đệm
<%@ include ... %>	Include một file trong suốt giai đoạn phiên dịch
<%@ taglib ... %>	Khai báo một thư viện thẻ, chứa các action tùy biến, được sử dụng trong trang đó

Bảng 4: 6 Các thẻ directive trong JSP

4.5.1. page Directive trong JSP

page Directive trong JSP được sử dụng để cung cấp các chỉ lệnh tới container mà liên quan tới JSP page hiện tại. Bạn có thể mã hóa page directive bất cứ đâu trong JSP page của bạn. Theo qui ước, page directive được mã hóa tại trên cùng của JSP page.

Sau đây là cú pháp cơ bản của page Directive trong JSP:

```
<%@ page attribute="value" %>
```

Bạn có thể viết XML tương đương với cú pháp trên như sau:

```
<jsp:directive.page attribute="value" />
```

Các thuộc tính của page directive trong JSP

Bảng dưới liệt kê các thuộc tính liên kết với page Directive trong JSP:

Thuộc tính	Mục đích
buffer	Xác định một mô hình đệm cho Output Stream
autoFlush	Điều khiển hành vi của Output buffer của Servlet
contentType	Định nghĩa giản đồ mã hóa ký tự
errorPage	Định nghĩa URL của JSP khác mà báo cáo trên Java các unchecked

	runtime exception
isErrorPage	Chỉ rằng nếu JSP page này là một URL được xác định bởi thuộc tính ErrorPage của JSP page khác
extends	Xác định một lớp cha mà Servlet đã tạo phải kế thừa
import	Xác định một danh sách các package hoặc class để sử dụng trong JSP khi lệnh import trong Java làm việc cho các lớp Java
info	Định nghĩa một chuỗi mà có thể được truy cập với phương thức getServletInfo() của Servlet
isThreadSafe	Định nghĩa mô hình thread cho Servlet đã tạo
language	Định nghĩa ngôn ngữ lập trình được sử dụng trong JSP page
session	Xác định có hay không JSP page tham dự trong các HTTP session
isELIgnored	Xác định có hay không EL expression bên trong JSP page sẽ bị bỏ qua
isScriptingEnabled	Quyết định nếu các phần tử scripting được cho phép sử dụng

Bảng 4: 7 Các thuộc tính liên kết với page Directive

Ta có thể hiểu chi tiết hơn về các thuộc tính trên tại: Page Directive trong JSP.

4.5.2. Include Directive trong JSP

include Directive trong JSP được sử dụng để bao một file trong suốt giai đoạn biên dịch. Directive này nói cho container sáp nhập content của file ngoại vi khác với JSP hiện tại trong quá trình biên dịch. Bạn có thể mã hóa include directive bất cứ đâu trong JSP page.

Dưới đây là form chung cho include directive trong JSP:

```
<%@ include file="relative url" >
```

Tên file trong include directive là một URL quan hệ thực sự. Nếu bạn chỉ xác định một tên file mà không có đường liên kết, thì JSP compiler giả sử rằng file đó là trong cùng thư mục như JSP của bạn.

Bạn có thể viết XML tương đương với cú pháp trên:

```
<jsp:directive.include file="relative url" />
```

Bạn có thể kiểm tra chi tiết về include directive tại: Include Directive trong JSP.

4.5.3. Taglib Directive trong JSP

Java Server Pages API cho phép bạn định nghĩa các Custom Tags trong JSP, mà giống như các thẻ HTML hoặc XML và một thư viện thẻ (tag library) là một tập hợp các thẻ được định nghĩa bởi người sử dụng mà triển khai các hành vi tùy biến.

taglib directive khai báo rằng: JSP page của bạn sử dụng một tập hợp các Custom Tags, thông báo vị trí của thư viện, và cung cấp một phương thức để nhận diện các Custom Tags đó trong JSP page của bạn.

Cú pháp của taglib directive trong JSP như sau:

```
<%@ taglib uri="uri" prefix="prefixOfTag" %>
```

Ở đây, giá trị thuộc tính uri biểu diễn một vị trí mà container có thể hiểu và thuộc tính prefix thông báo một container về các bit đánh dấu nào là các action tùy biến.

Bạn có thể viết XML tương đương với cú pháp trên như sau:

```
<jsp:directive.taglib uri="uri" prefix="prefixOfTag" />
```

4.6. Action trong JSP

JSP action sử dụng các chỉ lệnh trong cú pháp XML để điều khiển hành vi của Servlet engine. Bạn có thể chèn động một file, tái sử dụng các thành phần JavaBeans, chuyển người dùng tới trang khác, hoặc tạo HTML cho Java plugin.

Chỉ có một cú pháp cho phần tử Action trong JSP, khi nó tuân theo chuẩn XML.

```
<jsp:action_name attribute="value" />
```

Về cơ bản, các phần tử Action là các hàm tiền định nghĩa và có các action trong JSP có sẵn sau:

Cú pháp	Mục đích
jsp:include	Include một file tại thời điểm trang đó được yêu cầu
jsp:useBean	Tìm kiếm hoặc thuyết minh một JavaBean
jsp:setProperty	Thiết lập thuộc tính của một JavaBean

jsp:getProperty	Chèn thuộc tính của một JavaBean vào trong output
jsp:forward	Chuyển người yêu cầu tới trang mới
jsp:plugin	Tạo code cho trình duyệt riêng mà tạo một thẻ OBJECT hoặc EMBED cho Java plugin
jsp:element	Định nghĩa động các phần tử XML
jsp:attribute	Định nghĩa động thuộc tính của phần tử XML đã được định nghĩa
jsp:body	Định nghĩa động phần thân của phần tử XML đã được định nghĩa
jsp:text	Sử dụng để viết template text trong các JSP page và tài liệu JSP

Bảng 4: 8 Các Action trong JSP

Các thuộc tính phổ biến của action trong JSP

Có hai thuộc tính phổ biến cho các phần tử Action: thuộc tính id và thuộc tính scope:

Thuộc tính id: Thuộc tính này nhận diện một cách duy nhất phần tử Action, và cho phép action để được tham chiếu bên trong JSP page. Nếu Action tạo một đối tượng, thì giá trị id có thể được sử dụng để tham chiếu nó thông qua đối tượng ẩn PageContext.

Thuộc tính scope: Thuộc tính này nhận diện vòng đời của phần tử Action. Thuộc tính id và thuộc tính scope được liên quan trực tiếp, khi thuộc tính scope quyết định vòng đời của đối tượng mà liên kết với id. Thuộc tính scope có thể có 4 giá trị: page, request, session, và application.

4.6.1. <jsp:include> Action trong JSP

Action này cho phép bạn chèn các file vào trong các trang đang được tạo. Cú pháp như sau:

```
<jsp:include page="relative URL" flush="true" />
```

Không giống include directive, mà chèn file tại thời điểm JSP page được biên dịch thành một Servlet, include action này chèn file tại thời điểm page được yêu cầu.

Bảng sau liệt kê các thuộc tính liên quan với include action trong JSP:

Thuộc tính	Miêu tả
page	URL quan hệ của trang để được include
flush	Thuộc tính Boolean quyết định có hay không nguồn được bao có bộ đệm của nó được flush trước khi nó được bao

Bảng 4: 9 Các thuộc tính liên quan với include action trong JSP

Ví dụ ta định nghĩa hai file: date.jsp và main.jsp như sau:

Nội dung của date.jsp file:

```
<p>
    Today's date: <%= (new java.util.Date()).toLocaleString()%>
</p>
```

Nội dung của main.jsp file:

```
<html>
<head>
<title>The include Action Example</title>
</head>
<body>
<center>
<h2>The include action Example</h2>
<jsp:include page="date.jsp" flush="true" />
</center>
</body>
</html>
```

Giữ tất cả các file này trong thư mục gốc và thử truy cập main.jsp. Nó sẽ hiển thị kết quả như sau:

The include action Example

Today's date: 12-Sep-2010 14:54:22

4.6.2. `<jsp:useBean>` Action trong JSP

useBean action trong JSP là khá linh hoạt. Đầu tiên nó tìm kiếm một đối tượng đang tồn tại đang sử dụng thuộc tính id và scope. Nếu một đối tượng không được tìm thấy, thì nó cố gắng tạo đối tượng đã xác định đó.

Cách đơn giản nhất để tải một Bean như sau:

```
<jsp:useBean id="name" class="package.class" />
```

Khi lớp Bean đã được tải, bạn có thể sử dụng các **jsp:setProperty** action và **jsp:getProperty** action để sửa đổi và thu nhận các đặc tính của Bean.

Bảng dưới liệt kê các thuộc tính liên kết với useBean action trong JSP:

Thuộc tính	Miêu tả
class	Chỉ định tên package đầy đủ của Bean
type	Xác định kiểu biến mà sẽ tham chiếu tới đối tượng
beanName	Cung cấp tên của Bean như được xác định bởi phương thức instantiate() của lớp java.beans.Beans

Bảng 4: 10 Các thuộc tính liên kết với useBean action trong JSP

4.6.3. `<jsp:setProperty>` Action trong JSP

setProperty action trong JSP thiết lập các đặc tính của một Bean. Bean phải được tiên định nghĩa trước action này. Có hai cách cơ bản để sử dụng setProperty Action:

Bạn có thể sử dụng jsp:setProperty action sau khi, nhưng bên ngoài của một phần tử jsp:useBean như sau:

```
<jsp:useBean id="myName" ... />
...
<jsp:setProperty name="myName" property="someProperty" .../>
```

Trong trường hợp này, jsp:setProperty được thực thi, dù cho có hay không một Bean mới đã được thuyết minh hoặc một Bean đang tồn tại được tìm thấy.

Ngữ cảnh thứ hai mà jsp: setProperty có thể xuất hiện là bên trong phần thân của một phần tử jsp:useBean như sau:

```
<jsp:useBean id="myName" ... >
...
    <jsp:setProperty name="myName" property="someProperty" .../
>
```

`</jsp:useBean>`

Ở đây, `jsp:setProperty` được thực thi chỉ khi một đối tượng mới được tạo, không phải khi một đối tượng đang tồn tại được tìm thấy.

Bảng dưới liệt kê các thuộc tính liên kết với `setProperty` action trong JSP:

Thuộc tính	Miêu tả
name	Chỉ định Bean mà toàn bộ thuộc tính của nó sẽ được thiết lập. Bean phải được định nghĩa trước
property	Chỉ dẫn giá trị bạn muốn thiết lập. Một giá trị "*" nghĩa là tất cả tham số yêu cầu mà tên của nó so khớp với các tên property của Bean sẽ được truyền tới phương thức setter thích hợp
value	Giá trị mà để được gán tới property đã cho. Giá trị của tham số là null, hoặc tham số không tồn tại, <code>setProperty</code> bị bỏ qua
param	Thuộc tính param là tên của tham số yêu cầu mà có giá trị để được nhận. Bạn không thể sử dụng cả value và param

Bảng 4: 11 Các thuộc tính liên kết với `setProperty` action trong JSP

4.6.4. `<jsp:getProperty>` Action trong JSP

getProperty action trong JSP được sử dụng để thu nhận giá trị của một property đã cung cấp và biến đổi nó thành một chuỗi, và cuối cùng chèn nó vào trong output.

getProperty action chỉ có hai thuộc tính, cả hai thuộc tính này được yêu cầu như trong cú pháp đơn giản sau:

```
<jsp:useBean id="myName" ... />
...
<jsp:getProperty name="myName" property="someProperty" .../>
```

Sau đây là danh sách các thuộc tính liên kết với `getProperty` action trong JSP:

Thuộc tính	Miêu tả
name	Tên của Bean mà có một property để được thu nhận. Bean phải được định nghĩa trước

property	Thuộc tính property là tên của Bean property để được thu nhận
----------	---

Bảng 4: 12 Các thuộc tính liên kết với getProperty action trong JSP

Ví dụ

Sau đây ta định nghĩa một test Bean để sử dụng trong ví dụ sau:

```
/* File: TestBean.java */
package action;

public class TestBean {
    private String message = "No message specified";

    public String getMessage() {
        return(message);
    }

    public void setMessage(String message) {
        this.message = message;
    }
}
```

Biên dịch code trên để tạo TestBean.class file và bảo đảm rằng bạn đã sao TestBean.class trong C:\apache-tomcat-7.0.2\webapps\WEB-INF\classes\action và biến CLASSPATH cũng nên được thiết lập cho thư mục này.

Bây giờ sử dụng code trên trong main.jsp file mà tải Bean này và thiết lập/nhận một tham số String đơn giản:

```
<html>
<head>
<title>Using JavaBeans in JSP</title>
</head>
<body>
<center>
<h2>Using JavaBeans in JSP</h2>

<jsp:useBean id="test" class="action.TestBean" />
```

```

<jsp:setProperty name="test"
                  property="message"
                  value="Hello JSP..." />

<p>Got message....</p>

<jsp:getProperty name="test" property="message" />

</center>
</body>
</html>

```

Thử truy cập main.jsp, nó sẽ cho kết quả sau:

Using JavaBeans in JSP

```

Got message....

Hello JSP...

```

4.6.5. <jsp:forward> Action trong JSP

forward action trong JSP kết thúc action của trang hiện tại và chuyển request tới nguồn khác chẳng hạn như một Static page, JSP page khác, hoặc một Java Servlet.

Cú pháp đơn giản của forward action trong JSP là:

```
<jsp:forward page="Relative URL" />
```

Dưới đây là các tham số liên kết với forward action trong JSP:

Thuộc tính	Miêu tả
page	Nên bao gồm một URL quan hệ của nguồn khác chẳng hạn như một Static page, JSP page khác, hoặc một Java Servlet

Bảng 4: 13 Các tham số liên kết với forward action trong JSP

Ví dụ:

Ta tái sử dụng hai file là date.jsp và main.jsp như sau:

Sau đây là nội dung của date.jsp file:

```
<p>
```

```
Today's date: <%= (new java.util.Date()).toLocaleString()%>
</p>
```

Và nội dung của main.jsp file:

```
<html>
<head>
<title>The include Action Example</title>
</head>
<body>
<center>
<h2>The include action Example</h2>
<jsp:forward page="date.jsp" />
</center>
</body>
</html>
```

Bây giờ, bạn giữ các file này trong thư mục gốc và thử truy cập main.jsp. Nó sẽ cho kết quả như dưới đây. Nó loại bỏ nội dung từ main page và chỉ hiển thị nội dung từ trang chuyển tiếp tới.

```
Today's date: 12-Sep-2010 14:54:22
```

4.6.7. <jsp:plugin> Action trong JSP

Plugin action trong JSP được sử dụng để chèn các phần tử Java vào trong một JSP page. Nó chỉ định loại trình duyệt và chèn các thẻ <object> hoặc <embed> khi cần.

Nếu plugin cần đến là không có mặt, nó tải plugin và sau đó thực thi thành phần Java. Thành phần Java này có thể là một Applet hoặc một JavaBean.

Plugin action có một số thuộc tính mà tương ứng tới các thẻ HTML phổ biến được sử dụng để định dạng các thành phần Java. Phần tử <param> cũng có thể được sử dụng để gửi các tham số tới Applet hoặc Bean.

Cú pháp đặc trưng để sử dụng plugin action trong JSP như sau:

```
<jsp:plugin type="applet" codebase="dirname" code="MyApplet.class"
width="60" height="80">
  <jsp:param name="fontcolor" value="red" />
  <jsp:param name="background" value="black" />
```

```
<jsp:fallback>
    Unable to initialize Java Plugin
</jsp:fallback>
</jsp:plugin>
```

Bạn có thể thử action này bởi sử dụng một số Applet nếu bạn quan tâm. Một phần tử mới, phần tử `<fallback>`, có thể được sử dụng để xác định một Error string để được gửi tới người dùng trong trường hợp các thành phần đó lỗi.

4.6.7. `<jsp:element>`, `<jsp:attribute>`, `<jsp:body>`

Các `<jsp:element>`, `<jsp:attribute>` và `<jsp:body>` action trong JSP được sử dụng để định nghĩa động các phần tử XML. Từ động là quan trọng, bởi vì nó nghĩa là các phần tử XML có thể được tạo tại thời điểm request thay vì được định nghĩa một cách tĩnh tại thời điểm biên dịch.

Sau đây là cú pháp đơn giản để định nghĩa động các phần tử XML trong JSP:

```
<%@page language="java" contentType="text/html"%>
<html xmlns="http://www.w3c.org/1999/xhtml"
      xmlns:jsp="http://java.sun.com/jsp/Page">
<head><title>Generate XML Element</title></head>
<body>
<jsp:element name="xmlElement">
<jsp:attribute name="xmlElementAttr">
    Value for the attribute
</jsp:attribute>
<jsp:body>
    Body for XML element
</jsp:body>
</jsp:element>
</body>
</html>
```

Nó sẽ tạo HTML code sau tại runtime:

```
<html xmlns="http://www.w3c.org/1999/xhtml"
      xmlns:jsp="http://java.sun.com/jsp/Page">

<head><title>Generate XML Element</title></head>
```

```

<body>
  <xmlElement xmlElementAttr="Value for the attribute">
    Body for XML element
  </xmlElement>
</body>
</html>

```

4.6.8. <jsp:text> Action trong JSP

<jsp:text> action trong JSP có thể được sử dụng để viết Template text trong các JSP page và tài liệu JSP. Sau đây là cú pháp cho text action trong JSP:

```
<jsp:text>Template data</jsp:text>
```

Phần thân của Template không thể chứa bất kỳ phân tử nào; nó chỉ có thể chứa text và EL expression (EL expression được giải thích trong một chương sau đó). Bạn chú ý rằng, trong các XML file, bạn không thể sử dụng các Expression như `${whatever > 0}`, bởi vì ký hiệu lớn hơn là không được phép. Thay vào đó, sử dụng mẫu gt, giống như `${whatever gt 0}` hoặc một sự thay thế để được nhúng giá trị trong một CDATA section.

```
<jsp:text><![CDATA[<br>]]></jsp:text>
```

Nếu bạn cần bao một khai báo DOCTYPE, ví dụ cho XML, bạn cũng phải sử dụng phân tử <jsp:text> như sau:

```

<jsp:text><![CDATA[<!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
"DTD/xhtml11-strict.dtd">]]>
</jsp:text>
<head><title>jsp:text action</title></head>
<body>
  <books><book><jsp:text>
    Welcome to JSP Programming
  </jsp:text></book></books>
</body>
</html>

```

Bạn thử ví dụ sau khi sử dụng hoặc không sử dụng <jsp:text> action trong JSP.

4.7. Đối tượng ẩn (Implicit) trong JSP

Đối tượng ẩn (Implicit Object) trong JSP là các đối tượng Java mà JSP container làm nó có sẵn tới các lập trình viên trong mỗi trang và lập trình viên có thể gọi chúng một cách trực tiếp mà không phải khai báo rõ ràng. Đối tượng ẩn trong JSP cũng được gọi là các biến tiền định nghĩa.

JSP hỗ trợ 9 đối tượng ẩn sau đây:

Đối tượng	Miêu tả
request	Đây là đối tượng HttpServletRequest mà liên kết với Request
response	Đây là đối tượng HttpServletResponse mà liên kết với Response tới Client
out	Đây là đối tượng PrintWriter được sử dụng để gửi output tới Client
session	Đây là đối tượng HttpSession mà liên kết với Request
application	Đây là đối tượng ServletContext mà liên kết với application context
config	Đây là đối tượng ServletConfig mà liên kết với page
pageContext	Sự gói gọn này sử dụng các đặc trưng Server-Specific giống như hiệu năng cao JspWriters .
page	Được sử dụng để gọi các phương thức được định nghĩa bởi lớp Servlet đã được phiên dịch
Exception	Đối tượng Exception cho phép dữ liệu exception để được truy cập bởi JSP đã chỉ rõ

Bảng 4: 14 Các đối tượng ẩn trong JSP

4.7.1. Đối tượng request trong JSP

Đối tượng request trong JSP là một minh họa của đối tượng `javax.servlet.http.HttpServletRequest`. Mỗi khi một Client yêu cầu một trang, thì JSP engine tạo một đối tượng mới để biểu diễn yêu cầu đó.

Các đối tượng request cung cấp các phương thức để nhận thông tin HTTP header bao gồm Form data, cookie, phương thức HTTP, ...

Bạn sẽ thấy tập hợp toàn bộ các phương thức liên kết với đối tượng request trong chương: **Client Request trong JSP**.

4.7.2. Đối tượng response trong JSP

Đối tượng response là một minh họa của đối tượng

`javax.servlet.http.HttpServletResponse`. Ngay khi Server tạo đối tượng request, nó cũng tạo một đối tượng để biểu diễn phản hồi tới Client.

Đối tượng response cũng định nghĩa các Interface mà thực hiện việc tạo các HTTP header mới. Thông qua đối tượng này, lập trình viên JSP có thể thêm các Cookie mới hoặc Date, HTTP Status code, ...

Chúng ta sẽ bàn luận chi tiết về các phương thức liên kết với đối tượng response trong chương: **Server Response trong JSP**.

4.7.3. Đối tượng out trong JSP

Đối tượng out là đối tượng ẩn trong JSP và là một minh họa của đối tượng `javax.servlet.jsp.JspWriter`, được sử dụng để gửi content trong một phản hồi.

Đối tượng `JspWriter` ban đầu được khởi tạo một cách khác nhau phụ thuộc vào việc có hay không trang được đệm hoặc không. Có thể dễ dàng ngừng việc buffer này bởi sử dụng thuộc tính `buffered="false"` của page directive trong JSP.

Đối tượng `JspWriter` chứa hầu hết các phương thức giống như trong lớp `java.io.PrintWriter`. Tuy nhiên, `JspWriter` cũng có một số phương thức bổ sung được thiết kế để xử lý trình buffer. Không giống đối tượng `PrintWriter`, `JspWriter` ném `IOExceptions`.

Dưới đây là các phương thức quan trọng sẽ được sử dụng để viết Boolean, char, int, double, Object, String, ...

Phương thức	Miêu tả
out.print(dataType dt)	In một giá trị dataType
out.println(dataType dt)	In một giá trị dataType, sau đó kết thúc dòng với ký tự newline (dòng mới)
out.flush()	Flush luồng đó

Bảng 4: 15 Các phương thức của đối tượng out

4.7.3. Đối tượng session trong JSP

Đối tượng session là một minh họa của `javax.servlet.http.HttpSession` và vận hành hoàn toàn giống như cách đối tượng session hoạt động trong Java Servlet.

Đối tượng session được sử dụng để lần theo dấu Client session giữa các Client Request. Chúng ta sẽ thảo luận chi tiết cách sử dụng đối tượng session trong chương: **Session Tracking trong JSP**.

4.7.4. Đối tượng application trong JSP

Đối tượng application là bao quan trực tiếp đối tượng `ServletContext` cho Servlet đã được tạo và thực tế, nó là một minh họa của đối tượng `javax.servlet.ServletContext`.

Đối tượng này là một sự biểu diễn của JSP page thông qua cả vòng đời của nó. Đối tượng application được tạo khi JSP page được khởi tạo và sẽ được gỡ bỏ khi JSP page bị gỡ bỏ bởi phương thức `jspDestroy()`.

Bằng việc thêm một thuộc tính tới đối tượng application, bạn có thể bảo đảm rằng tất cả JSP file, mà tạo ứng dụng web của bạn, có quyền truy cập tới nó.

Bạn xem xét sự sử dụng của đối tượng application trong: **Hit counter trong JSP**.

4.7.5. Đối tượng config trong JSP

Đối tượng config là một sự thuyết minh của đối tượng `javax.servlet.ServletConfig` và là bao quanh trực tiếp đối tượng `ServletConfig` cho Servlet đã được tạo.

Đối tượng này cho phép lập trình viên JSP truy cập tới các tham số khởi tạo Servlet engine hoặc JSP engine, chẳng hạn như path hoặc vị trí file, ...

Đây là cú pháp của phương thức config trong JSP:

```
config.getServletName();
```

Nó trả về một tên Servlet, là một chuỗi được chứa trong phần tử `<servlet-name>` được định nghĩa trong `WEB-INF\web.xml` file.

4.7.6. Đối tượng pageContext trong JSP

Đối tượng `pageContext` là một minh họa của đối tượng `javax.servlet.jsp.PageContext`. Đối tượng này được sử dụng để biểu diễn toàn bộ JSP page.

Đối tượng `pageContext` trong JSP như là một phương thức để truy cập thông tin về trang trong khi tránh xa hầu hết các chi tiết về sự triển khai.

Đối tượng này lưu giữ các tham chiếu tới các đối tượng request và response cho mỗi yêu cầu. Các đối tượng application, config, session, và out được nội suy bởi truy cập các thuộc tính của đối tượng này.

Đối tượng `pageContext` cũng chứa thông tin về các directive mà được thông báo tới JSP page, bao gồm thông tin đệm, `errorPageURL` và phạm vi (scope) trang.

Lớp `PageContext` định nghĩa một số các trường, bao gồm `PAGE_SCOPE`, `REQUEST_SCOPE`, `SESSION_SCOPE`, và `APPLICATION_SCOPE`, mà nhận diện 4 phạm vi. Nó cũng hỗ trợ hơn 40 phương thức, trong đó khoảng một nửa là kế thừa từ lớp `javax.servlet.jsp.JspContext`.

Một trong những phương thức quan trọng là **`removeAttribute`**, mà chấp nhận một hoặc hai tham số. Ví dụ, `pageContext.removeAttribute("attrName")` gỡ bỏ thuộc tính từ tất cả scope, trong khi code sau chỉ gỡ bỏ nó từ page scope.

```
pageContext.removeAttribute("attrName", PAGE_SCOPE);
```

4.7.7. Đối tượng page trong JSP

Đối tượng page là một tham chiếu thực sự tới page. Nó có thể được xem như một đối tượng mà biểu diễn toàn bộ JSP page.

Đối tượng Exception trong JSP

Đối tượng Exception là một wrapper chứa Exception được ném từ page trước. Đặc trưng, nó được sử dụng để tạo một phản hồi thích hợp tới điều kiện lỗi đó.

Câu hỏi ôn tập.

1. Cấu Trúc Cơ Bản của Một Trang JSP là Gì?. Mô tả cấu trúc cơ bản của một trang JSP và các phần quan trọng.
2. Vòng Đời của Một Trang JSP Bao Gồm Những Gì?. Liệt kê các giai đoạn trong vòng đời của một trang JSP và giải thích ý nghĩa của chúng.
3. Đối Tượng Request là Gì Trong JSP?. Mô tả vai trò và sử dụng của đối tượng request trong JSP.
4. Làm Thế Nào Để Sử Dụng Biến Được Chuyển Từ Servlet Sang JSP?. Giải thích cách sử dụng biến được chuyển từ Servlet sang JSP và cách truy cập chúng.
5. Các Đối Tượng Ẩn Trong JSP Là Gì?. Liệt kê và mô tả vai trò của các đối tượng ẩn như request, response, và session.

Bài 8: Lập trình JSP(Tiếp theo).

Trong phần tiếp theo chúng ta sẽ tìm hiểu các thành phần quan trọng khác trong JSP như: Request, Response và xử lý Form trong JSP.

4.8. Client Request trong JSP

Khi một trình duyệt yêu cầu một Webpage, nó gửi nhiều thông tin tới Web server, mà không thể được đọc một cách trực tiếp bởi vì thông tin này là một phần trong Header của HTTP request. Bạn có thể truy cập chương: Giao thức HTTP để theo dõi chi tiết về điều này.

Dưới đây là các thông tin header quan trọng mà tới từ trình duyệt mà ta sẽ sử dụng thường xuyên trong lập trình web.

Header	Miêu tả
Accept	Trường này xác định loại MIME mà trình duyệt hoặc Client khác có thể xử lý. Các giá trị image/png hoặc image/jpeg là hai khả năng phổ biến nhất
Accept-Charset	Trường này xác định bộ ký tự mà trình duyệt có thể sử dụng để hiển thị thông tin. Ví dụ: ISO-8859-1
Accept-Encoding	Trường này xác định kiểu mã hóa mà trình duyệt biết để xử lý. Các giá trị của gzip hoặc compress là hai khả năng phổ biến nhất
Accept-Language	Trường này xác định ngôn ngữ ưa thích của Client trong trường hợp Servlet có thể tạo các kết quả trong nhiều ngôn ngữ. Ví dụ en, en-us, ru, .v.v.
Authorization	Trường này được sử dụng bởi Client để nhận diện chính nó khi truy cập vào các trang web được bảo vệ bởi password
Connection	Trường này chỉ dẫn có hay không Client có thể xử lý các kết nối HTTP dạng Persistent. Các kết nối Persistent cho phép Client hoặc trình duyệt thu nhận nhiều file với một yêu cầu đơn. Một giá trị của Keep-Alive nghĩa là các kết nối Persistent này nên được sử dụng
Content-Length	Trường này chỉ thích hợp với các yêu cầu POST và cung cấp kích cỡ của dữ liệu POST bằng giá trị byte
Cookie	Trường này trả về các cookie tới các Server mà trước đó đã gửi chúng tới trình duyệt
Host	Trường này xác định host và cổng (port) khi được cung cấp trong URL ban đầu
If-Modified-Since	Trường này chỉ rằng Client muốn trang này chỉ khi nó đã được thay đổi sau ngày đã xác định. Server gửi một code là 304, nghĩa là Not Modified Header nếu không có kết quả nào mới hơn là có sẵn

If-Unmodified-Since	Trường này là ngược với trường If-Modified-Since. Nó xác định hoạt động là thành công chỉ khi tài liệu là cũ hơn ngày đã xác định
Referer	Trường này chỉ URL của trang web đang tham chiếu đó. Ví dụ, nếu bạn tại trang Webpage 1 và click vào một link tới trang Webpage 2, thì URL của Webpage 1 được bao trong trường Referer Header khi trình duyệt yêu cầu trang Webpage 2
User-Agent	Trường này nhận diện trình duyệt hoặc Client đang tạo yêu cầu và có thể được sử dụng để trả về nội dung khác nhau cho các loại trình duyệt khác nhau

Bảng 4: 16 Các thông tin header từ trình duyệt

Đối tượng HttpServletRequest trong JSP

Đối tượng request là một minh họa của một đối tượng `javax.servlet.http.HttpServletRequest`. Mỗi khi một Client yêu cầu một trang, thì JSP engine tạo một đối tượng mới để biểu diễn yêu cầu đó.

Đối tượng request cung cấp các phương thức để nhận thông tin HTTP header gồm Form data, cookie, phương thức HTTP, ...

Bảng dưới liệt kê các phương thức quan trọng có thể được sử dụng để đọc HTTP header trong chương trình JSP. Các phương thức này có sẵn với đối tượng `HttpServletRequest` mà biểu diễn Client Request tới Webserver.

STT	Phương thức & Miêu tả
1	Cookie[] getCookies(): Trả về một mảng chứa tất cả đối tượng Cookie mà Client gửi với yêu cầu này
2	Enumeration getAttributeNames(): Trả về một Enumeration chứa các tên của thuộc tính có sẵn cho yêu cầu này
3	Enumeration getHeaderNames(): Trả về một Enumeration của tất cả tên Header mà yêu cầu này chứa
4	Enumeration getParameterNames(): Trả về một Enumeration của các đối tượng String chứa tên của Parameter được chứa trong yêu cầu này

5	HttpSession getSession(): Trả về session hiện tại, mà liên kết với yêu cầu này, hoặc yêu cầu đó không có một session, nó sẽ tạo ra một session
6	HttpSession getSession(boolean create): Trả về HttpSession hiện tại, mà liên kết với yêu cầu này, hoặc nếu không có session hiện tại nào và create là true, nó trả về một session mới
7	Locale getLocale(): Trả về Locale mà Client sẽ chấp nhận nội dung trong đó, dựa trên trường Accept-Language Header
8	Object getAttribute(String name): Trả về giá trị của thuộc tính name như là một Object, hoặc trả về null nếu không có thuộc tính nào trong name đã cung cấp tồn tại
9	ServletInputStream getInputStream(): Thu nhận thân yêu cầu dạng dữ liệu nhị phân bởi sử dụng một ServletInputStream
10	String getAuthType() Trả về tên của Authentication Scheme được sử dụng để bảo vệ Servlet, ví dụ: "BASIC" hoặc "SSL", hoặc trả về null nếu JSP không được bảo vệ
11	String getCharacterEncoding(): Trả về tên của Mã hóa ký tự được sử dụng trong thân yêu cầu này
12	String getContentType(): Trả về kiểu MIME của thân yêu cầu, hoặc trả về null nếu không biết kiểu này
13	String getContextPath(): Trả về phần URI yêu cầu mà chỉ dẫn context của yêu cầu đó
14	String getHeader(String name): Trả về giá trị của trường Header đã xác định dưới dạng một String
15	String getMethod(): Trả về tên của phương thức HTTP mà yêu cầu này

	được tạo, ví dụ: GET, POST, hoặc PUT
16	String getParameter(String name): Trả về giá trị của Parameter dưới dạng một String, hoặc trả về null nếu Parameter này không tồn tại
17	String getPathInfo(): Trả về bất kỳ thông tin path bổ sung, mà liên kết với URI mà Client đã gửi khi nó tạo yêu cầu này
18	String getProtocol(): Trả về tên và phiên bản của giao thức
19	String getQueryString(): Trả về chuỗi truy vấn, được chứa trong URL sau path đó
20	String getRemoteAddr(): Trả về địa chỉ Internet Protocol (IP) của một Client mà gửi yêu cầu này
21	String getRemoteHost(): Trả về tên đầy đủ của Client mà gửi yêu cầu này
22	String getRemoteUser() Trả về login của người sử dụng tạo yêu cầu này, nếu người dùng đã được xác nhận, hoặc trả về null nếu người dùng chưa được xác nhận
23	String getRequestURI(): Trả về phần URL của yêu cầu này từ tên giao thức dưới dạng một chuỗi truy vấn trong dòng đầu tiên của HTTP Request
24	String getRequestedSessionId(): Trả về ID của session đã được xác định bởi Client
25	String getServletPath(): Trả về phần URL của yêu cầu này mà gọi JSP
26	String[] getParameterValues(String name): Trả về một mảng đối tượng String chứa tất cả các giá trị mà các parameter đã cung cấp có, hoặc trả về null nếu parameter này không tồn tại

27	boolean isSecure(): Trả về một Boolean chỉ rằng có hay không yêu cầu này được tạo bởi sử dụng một đường an toàn, như HTTPS
28	int getContentLength(): Trả về độ dài, bằng giá trị byte, của thân yêu cầu và làm nó có sẵn bởi Input Stream, hoặc trả về -1 nếu độ dài là không được biết
29	int getIntHeader(String name): Trả về giá trị của Request header đã cho dưới dạng một int
30	int getServerPort(): Trả về số hiệu cổng mà trên đó yêu cầu được nhận

Bảng 4: 17 Các phương thức của HttpServletRequest

Ví dụ HTTP Header Request trong JSP

Ví dụ sau sử dụng phương thức `getHeaderName()` của đối tượng `HttpServletRequest` để đọc thông tin HTTP header. Phương thức này trả về một Enumeration chứa thông tin header mà liên kết với HTTP request hiện tại.

Khi chúng ta có một Enumeration, chúng ta có thể lặp Enumeration này theo phương thức chuẩn: sử dụng phương thức `hasMoreElements()` để xác định khi nào dừng và sử dụng phương thức `nextElement()` để nhận tên mỗi parameter.

```
<%@ page import="java.io.*,java.util.*" %>
<html>
<head>
<title>HTTP Header Request Example</title>
</head>
<body>
<center>
<h2>HTTP Header Request Example</h2>
<table width="100%" border="1" align="center">
<tr bgcolor="#949494">
<th>Header Name</th><th>Header Value(s)</th>
</tr>
<%
    Enumeration headerNames = request.getHeaderNames();
    while(headerNames.hasMoreElements()) {
```

```

        String paramName = (String)headerNames.nextElement();
        out.print("<tr><td>" + paramName + "</td>\n");
        String paramValue = request.getHeader(paramName);
        out.println("<td> " + paramValue + "</td></tr>\n");
    }
%>
</table>
</center>
</body>
</html>

```

Bây giờ đặt code trên trong main.jsp và thử truy cập nó. Nó sẽ cho kết quả sau:
 Tên và giá trị tương ứng của HTTP Header Request trong JSP

Tên Header	Giá trị Header
accept	*/*
accept-language	en-us
user-agent	Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 5.1; Trident/4.0; InfoPath.2; MS-RTC LM 8)
accept-encoding	gzip, deflate
host	localhost:8080
connection	Keep-Alive
cache-control	no-cache

Bảng 4: 18 Tên và giá trị tương ứng của HTTP Header Request trong JSP

Để làm quen với các phương thức này, ta thử viết các chương trình sử dụng các phương thức trên theo cách thức tương tự trên.

4.9. Server Response trong JSP

Khi một Web Server phản hồi một HTTP Request tới trình duyệt, phản hồi đặc trưng bao gồm một dòng Status, một số trường Header, một dòng trống, và tài liệu. Một phản hồi đặc trưng giống như sau:

```

HTTP/1.1 200 OK
Content-Type: text/html
Header2: ...
...

```

```

HeaderN: ...
(Blank Line)
<!doctype ...>
<html>
<head>...</head>
<body>
...
</body>
</html>

```

Dòng Status gồm HTTP version (trong ví dụ là HTTP/1.1), một Status code (trong ví dụ là 200), và một thông báo rất ngắn tương ứng với Status code (trong ví dụ là OK).

Dưới đây là các trường Header hữu ích nhất trong phản hồi HTTP 1.1 mà trở lại trình duyệt từ Web Server và ta sẽ thường xuyên sử dụng chúng trong lập trình web:

Header	Miêu tả
Allow	Trường này xác định các phương thức yêu cầu (GET, POST, .v.v.) mà Server hỗ trợ
Cache-Control	Trường này xác định các hoàn cảnh mà trong đó tài liệu phản hồi có thể được cache một cách an toàn. Nó có thể có các giá trị public , private hoặc no-cache , .v.v.. Trong đó, public nghĩa là tài liệu có thể được cache, private nghĩa là tài liệu dành cho một người sử dụng và chỉ có thể được lưu giữ riêng (không được chia sẻ) và no-cache nghĩa là tài liệu không bao giờ được cache
Connection	Trường này chỉ dẫn trình duyệt có sử dụng persistent trong các kết nối HTTP hoặc không. Một giá trị close chỉ rằng trình duyệt không sử dụng các kết nối HTTP dạng persistent và keep-alive nghĩa là sử dụng các kết nối persistent
Content-Disposition	Trường này cho bạn yêu cầu rằng trình duyệt hỏi người sử dụng để lưu phản hồi lên đĩa trong một file với tên đã cho
Content-Encoding	Trường này xác định cách mà trang được mã hóa trong khi truyền tải

Content-Language	Trường này xác định ngôn ngữ mà tài liệu được viết. Ví dụ en, en-us, ru, ...
Content-Length	Trường này xác định số byte trong phản hồi. Thông tin này chỉ cần thiết nếu trình duyệt đang sử dụng một kết nối HTTP dạng persistent (giá trị keep-alive)
Content-Type	Trường này kiểu MIME (Multipurpose Internet Mail Extension) của tài liệu phản hồi
Expires	Trường này xác định thời gian tại đó nội dung nên được xem là hết hạn và không bao giờ được cache nữa
Last-Modified	Trường này chỉ dẫn khi tài liệu được thay đổi lần cuối. Sau đó Client có thể cache tài liệu này và cung cấp một date bởi một trường yêu cầu If-Modified-Since trong các yêu cầu sau đó
Location	Trường này nên được bao với tất cả phản hồi mà có Status code là 300. Nó thông báo cho trình duyệt về địa chỉ tài liệu. Trình duyệt tự động kết nối lại tới vị trí này và thu nhận tài liệu mới
Refresh	Trường này xác định khi nào trình duyệt nên đề nghị một trang đã được update. Bạn có thể xác định thời gian với đơn vị là số giây sau đó một trang sẽ được refresh
Retry-After	Trường này có thể được sử dụng khi kết hợp với phản hồi 503 (Service Unavailable) để nói cho Client khi nào thì nó có thể lặp lại yêu cầu
Set-Cookie	Trường này xác định một cookie mà liên kết với trang

Bảng 4: 19 Các trường Header từ trình duyệt

Đối tượng HttpServletResponse trong JSP

Đối tượng response là một minh họa của đối tượng

javax.servlet.http.HttpServletResponse. Ngay khi Server tạo đối tượng request, nó cũng tạo một đối tượng để biểu diễn phản hồi tới Client.

Đối tượng Response cũng định nghĩa các Interface mà thực hiện việc tạo các HTTP header mới. Thông qua đối tượng này, lập trình viên JSP có thể thêm các Cookie mới hoặc Date, HTTP Status code, ...

Dưới đây là các phương thức có thể được sử dụng để thiết lập HTTP response Header trong chương trình JSP của bạn. Những phương thức này có sẵn với đối tượng **HttpServletResponse** mà biểu diễn Server Response.

STT	Phương thức và Miêu tả
1	String encodeRedirectURL(String url) Mã hóa URL đã cho để sử dụng trong phương thức sendRedirect, hoặc nếu mã hóa này không được đòi hỏi, thì trả về URL như cũ
2	String encodeURL(String url) Mã hóa URL đã cho bằng việc thêm ID của session trong nó, hoặc nếu mã hóa này không được đòi hỏi, thì trả về URL như cũ
3	boolean containsHeader(String name) Trả về một Boolean chỉ rằng có hay không header phản hồi với tên đã cho đã được thiết lập
4	boolean isCommitted() Trả về một Boolean chỉ rằng nếu phản hồi đã được commit
5	void addCookie(Cookie cookie) Thêm cookie đã cho tới phản hồi
6	void addDateHeader(String name, long date) Thêm một header phản hồi với tên và giá trị date đã cho
7	void addHeader(String name, String value) Thêm một header phản hồi với tên và giá trị đã cho
8	void addIntHeader(String name, int value) Thêm một header phản hồi với tên và giá trị integer đã cho
9	void flushBuffer()

	Force bắt kỳ nội dung nào trong Buffer để được ghi tới Client đó
10	void reset() Xóa bất dữ liệu nào tồn tại trong buffer cũng như Status code và các trường header
11	void resetBuffer() Xóa nội dung của buffer nằm dưới trong phản hồi mà không xóa các trường header và status code
12	void sendError(int sc) Gửi một phản hồi lỗi tới Client bởi sử dụng Status code đã cho và xóa buffer
13	void sendError(int sc, String msg) Gửi một phản hồi lỗi tới Client bởi sử dụng Status code đã cho
14	void sendRedirect(String location) Gửi một phản hồi Redirect tạm thời tới Client bởi sử dụng địc chỉ URL để Redirect đã cho
15	void setBufferSize(int size) Thiết lập kích cỡ buffer cho thân phản hồi
16	void setCharacterEncoding(String charset) Thiết lập mã hóa ký tự (MIME Charset) của phản hồi đang được gửi tới Client, ví dụ, thiết lập là UTF-8
17	void setContentLength(int len) Thiết lập độ dài nội dung của thân phản hồi trong HTTP Servlet, phương thức này thiết lập trường header là HTTP Content-Length
18	void setContentType(String type) Thiết lập kiểu nội dung của phản hồi đang được gửi tới Client, nếu phản hồi vẫn chưa được commit
19	void setDateHeader(String name, long date) Thiết lập một header phản hồi với tên và giá trị date đã cho

20	void setHeader(String name, String value) Thiết lập một header phản hồi với tên và giá trị đã cho
21	void setIntHeader(String name, int value) Thiết lập một header phản hồi với tên và giá trị integer đã cho
22	void setLocale(Locale loc) Thiết lập Locale của phản hồi, nếu phản hồi vẫn chưa được commit
23	void setStatus(int sc) Thiết lập Status code cho phản hồi này

Bảng 4: 20 Các phương thức của HttpServletResponse trong JSP

Ví dụ về HTTP Header Response trong JSP

Ví dụ sau sẽ sử dụng phương thức `setIntHeader()` để thiết lập Refresh Header để đóng vai một digital clock.

```
<%@ page import="java.io.*,java.util.*" %>
<html>
<head>
<title>Auto Refresh Header Example</title>
</head>
<body>
<center>
<h2>Auto Refresh Header Example</h2>
<%
    // Set refresh, autoloading time as 5 seconds
    response.setIntHeader("Refresh", 5);
    // Get current time
    Calendar calendar = new GregorianCalendar();
    String am_pm;
    int hour = calendar.get(Calendar.HOUR);
    int minute = calendar.get(Calendar.MINUTE);
    int second = calendar.get(Calendar.SECOND);
    if(calendar.get(Calendar.AM_PM) == 0)
        am_pm = "AM";
```

```

else
    am_pm = "PM";
String CT = hour+":"+ minute + ":" + second + " " + am_pm;
out.println("Current Time is: " + CT + "\n");
%>
</center>
</body>
</html>

```

Bây giờ, bạn đặt code trên trong main.jsp và thử truy cập nó. Nó sẽ hiển thị System time sau mỗi 5s. Chạy JSP và đợi kết quả:

Auto Refresh Header Example

Current Time is: 9:44:50 PM

Để làm quen với các phương thức này, bạn thử viết các chương trình sử dụng các phương thức trên theo cách thức tương tự trên.

4.10. Status Code trong JSP

Định dạng của các thông báo HTTP request và HTTP response là tương tự nhau và sẽ có cấu trúc sau:

- Một dòng status + CRLF (Carriage Return + Line Feed ví dụ: New Line)
- 0 hoặc nhiều dòng header + CRLF
- Một dòng trống, ví dụ: một CRLF
- Một thân thông báo tùy ý như file, dữ liệu truy vấn (Query Data) hoặc kết quả truy vấn (Query output).

Ví dụ, một header phản hồi từ Server trông giống như sau:

```

HTTP/1.1 200 OK
Content-Type: text/html
Header2: ...
...
HeaderN: ...
(Blank Line)
<!doctype ...>
<html>
<head>...</head>

```

```
<body>
...
</body>
</html>
```

Dòng Status gồm phiên bản HTTP (trong ví dụ là HTTP/1.1), một status code (trong ví dụ là 200), và một thông báo rất ngắn tương ứng với Status code đó (trong ví dụ là OK).

Bảng dưới liệt kê các HTTP Status code và các thông báo tương ứng mà có thể được trả về từ Web Server:

Code	Thông báo	Miêu tả
100	Continue	Chỉ một phần của yêu cầu đã được nhận bởi Server, nhưng chỉ cần nó không bị loại bỏ, Client nên tiếp tục với yêu cầu này
101	Switching Protocols	Server chuyển đổi giữa các giao thức
200	OK	Yêu cầu là OK
201	Created	Yêu cầu là hoàn thành, và một nguồn mới được tạo
202	Accepted	Yêu cầu được chấp nhận để xử lý, nhưng việc xử lý chưa hoàn thành
203	Non-authoritative Information	
204	No Content	
205	Reset Content	
206	Partial Content	

300	Multiple Choices	Một danh sách các link. Người dùng có thể chọn một link và tới trang đó. Tối đa là 5 địa chỉ
301	Moved Permanently	Trang được yêu cầu đã chuyển tới một URL mới
302	Found	Trang được yêu cầu đã chuyển tới một URL mới
303	See Other	Trang được yêu cầu có thể được tìm với một URL khác
304	Not Modified	
305	Use Proxy	
306	Unused	Code này được sử dụng trong một phiên bản trước. Nó không còn được sử dụng nữa, nhưng vẫn được lưu trữ
307	Temporary Redirect	Trang được yêu cầu đã tạm thời chuyển tới một URL mới
400	Bad Request	Server không hiểu yêu cầu
401	Unauthorized	Trang được yêu cầu cần thiết một username và một password
402	Payment Required	<i>Bạn không thể sử dụng code này nữa</i>
403	Forbidden	Sự truy cập là bị cấm với trang được yêu cầu
404	Not Found	Server không tìm thấy trang được yêu cầu
405	Method Not Allowed	Phương thức đã được xác định trong yêu cầu là không được cho phép
406	Not	Server chỉ có thể tạo một phản hồi mà không được chấp

	Acceptable	nhận bởi Client
407	Proxy Authentication Required	Bạn phải xác nhận với một Proxy server trước khi yêu cầu này có thể được phục vụ
408	Request Timeout	Yêu cầu tốn thời gian quá lâu so với thời gian server đợi
409	Conflict	Yêu cầu này không thể hoàn thành bởi vì có một conflict
410	Gone	Trang được yêu cầu không có sẵn nữa
411	Length Required	"Content-Length" không được định nghĩa. Server sẽ không chấp nhận yêu cầu mà không có nó
412	Precondition Failed	Điều kiện tiên quyết (precondition) đã cung cấp trong yêu cầu được tính toán là false bởi Server
413	Request Entity Too Large	Server sẽ không chấp nhận yêu cầu này, bởi vì đối tượng yêu cầu là quá lớn
414	Request-url Too Long	Server sẽ không chấp nhận yêu cầu này, bởi vì URL là quá dài. Xảy ra khi bạn biến đổi một yêu cầu POST thành một yêu cầu GET với một thông tin truy vấn quá dài
415	Unsupported Media Type	Server sẽ không chấp nhận yêu cầu, bởi vì MIME type là không được hỗ trợ
417	Expectation Failed	
500	Internal Server Error	Yêu cầu chưa được hoàn thành. Server gặp một trường hợp không mong muốn
501	Not Implemented	Yêu cầu chưa được hoàn thành. Server không hỗ trợ tính năng được yêu cầu

502	Bad Gateway	Yêu cầu chưa được hoàn thành. Server này nhận một phản hồi không hợp lệ từ Server tuyến trên
503	Service Unavailable	Yêu cầu chưa được hoàn thành. Server tạm thời đang quá tải hoặc hỏng
504	Gateway Timeout	Gateway là hết hạn
505	HTTP Version Not Supported	Server không hỗ trợ phiên bản "http protocol"

Bảng 4: 21 Các HTTP Status code và các thông báo tương ứng

Phương thức để thiết lập HTTP Status Code

Dưới đây liệt kê các phương thức có thể được sử dụng để thiết lập HTTP Status code trong chương trình JSP. Các phương thức này có sẵn với đối tượng

HttpServletResponse.

TT	Phương thức & Miêu tả
1	public void setStatus (int statusCode) Phương thức này thiết lập một Status code tùy ý. Phương thức setStatus nhận một int (là status code) như là một tham số. Nếu phản hồi của bạn bao gồm một Status code đặc biệt và một tài liệu, đảm bảo bạn đã gọi setStatus trước khi thực sự trả về bất kỳ nội dung vào với <i>PrintWriter</i>
2	public void sendRedirect(String url) Phương thức này tạo một phản hồi là 302 cùng với một <i>Location</i> header cung cấp URL của tài liệu mới đó
3	public void sendError(int code, String message) Phương thức này gửi một Status code (thường là 404) cùng với một thông báo ngắn mà tự động được định dạng bên trong một tài liệu HTML và được gửi tới Client

Bảng 4: 22 Các phương thức sử dụng để thiết lập HTTP Status code trong chương trình JSP

Ví dụ về HTTP Status Code

Ví dụ sau sẽ gửi code lỗi là 407 tới trình duyệt Client và trình duyệt sẽ hiển thị thông báo "Need authentication".

```
<html>
<head>
<title>Setting HTTP Status Code</title>
</head>
<body>
<%
    // Set error code and reason.
    response.sendError(407, "Need authentication!!!" );
%>
</body>
</html>
```

Bây giờ, bạn gọi JSP trên sẽ hiển thị kết quả sau:

HTTP Status 407 - Need authentication!!!

type Status report

message Need authentication!!!

Miêu tả The client must first authenticate itself with the proxy (Need authentication!!!).

Apache Tomcat/5.5.29

Để làm quen với các phương thức trên, bạn thử viết các chương trình sử dụng các phương thức được liệt kê ở trên theo cách thức tương tự trong ví dụ này.

4.11. Xử lý Form trong JSP

Ta có thể gặp một vài tình huống khi bạn cần truyền một số thông tin từ trình duyệt của bạn tới Webserver và sau cùng tới chương trình backend của bạn. Trình duyệt sử dụng hai phương thức để truyền thông tin này tới Webserver. Các phương thức này là phương thức GET và phương thức POST.

4.11.1. Phương thức GET

Phương thức GET gửi thông tin người dùng đã được mã hóa phụ thêm tới yêu cầu trang. Trang này và thông tin đã được mã hóa được phân biệt bởi ký tự ? như sau:

```
http://www.test.com/hello?key1=value1&key2=value2
```

Phương thức GET là phương thức mặc định để truyền thông tin từ trình duyệt tới Web Server và nó tạo ra một chuỗi dài mà xuất hiện trong Location:box của trình duyệt. Không bao giờ sử dụng phương thức GET nếu bạn truyền tới Server thông tin password hoặc thông tin nhạy cảm khác.

Phương thức GET có kích cỡ giới hạn: chỉ có 1024 ký tự trong một chuỗi yêu cầu.

Thông tin này được truyền bởi sử dụng QUERY_STRING Header và sẽ có thể truy cập thông qua biến môi trường QUERY_STRING mà có thể được xử lý bởi sử dụng phương thức `getQueryString()` và `getParameter()` của đối tượng request trong JSP.

4.11.2. Phương thức POST

Một phương thức đáng tin cậy hơn để truyền thông tin tới chương trình backend là phương thức POST.

Nó gói thông tin theo cách giống như phương thức GET, nhưng thay vì gửi nó như là một chuỗi text sau một ký tự ? trong URL, nó gửi thông tin như là một thông báo riêng biệt. Thông báo này tới chương trình backend trong form của input chuẩn mà bạn có thể parse và sử dụng cho tiến trình xử lý của bạn.

JSP xử lý kiểu yêu cầu này bởi sử dụng phương thức **`getParameter()`** để đọc các tham số đơn giản và phương thức **`getInputStream()`** để đọc các luồng dữ liệu nhị phân tới từ Client.

Đọc Form Data bởi sử dụng JSP

JSP xử lý Form Data bằng việc parse một cách tự động bởi sử dụng các phương thức sau tùy thuộc vào từng tình huống cụ thể:

- **`getParameter()`**: Bạn gọi phương thức `request.getParameter()` để nhận giá trị của tham số Form.
- **`getParameterValues()`**: Gọi phương thức này nếu parameter xuất hiện nhiều hơn một lần và trả về nhiều giá trị, ví dụ như checkbox.
- **`getParameterNames()`**: Gọi phương thức này nếu bạn muốn một danh sách đầy đủ của tất cả các parameter trong yêu cầu hiện tại.
- **`getInputStream()`**: Gọi phương thức này để đọc luồng dữ liệu nhị phân tới từ Client.

Ví dụ phương thức GET sử dụng URL trong JSP

Đây là một URL đơn giản mà sẽ truyền hai giá trị tới chương trình HelloForm bởi sử dụng phương thức GET:

http://localhost:8080/main.jsp?first_name=ZARA&last_name=ALI

Dưới đây là chương trình JSP tên là main.jsp để xử lý input được cung cấp bởi trình duyệt web. Chúng ta đang sử dụng phương thức **getParameter()** để truy cập các thông tin đã truyền dễ dàng hơn.

```
<html>
<head>
<title>Using GET Method to Read Form Data</title>
</head>
<body>
<center>
<h1>Using GET Method to Read Form Data</h1>
<ul>
<li><p><b>First Name:</b>
    <%= request.getParameter("first_name")%>
</p></li>
<li><p><b>Last Name:</b>
    <%= request.getParameter("last_name")%>
</p></li>
</ul>
</body>
</html>
```

Kết quả khi truy vấn

http://localhost:8080/main.jsp?first_name=ZARA&last_name=ALI trong Location:box của trình duyệt. Nó sẽ cho kết quả sau:

Using GET Method to Read Form Data

First Name: ZARA

Last Name: ALI

Ví dụ phương thức GET sử dụng Form trong JSP

Ví dụ đơn giản sau sẽ truyền hai giá trị bởi sử dụng HTML FORM và nút submit. Chúng ta cũng đang sử dụng chương trình JSP là main.jsp để xử lý input này.

```
<html>
<body>
```

```
<form action="main.jsp" method="GET">
First Name: <input type="text" name="first_name">
<br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>
</body>
</html>
```

Giữ HTML này trong một Hello.htm file và đặt nó trong thư mục <Tomcat-installation-directory>/webapps/ROOT. Khi bạn truy cập

http://localhost:8080/Hello.htm, thì đây là output của form trên.

First Name:

First Name:

Bạn thử nhập First Name và Last Name và sau đó vào nút submit để xem kết quả trên thiết bị nội bộ của bạn, nơi Tomcat đang chạy. Dựa trên input đã cung cấp, nó sẽ tạo ra cùng kết quả như đã nói trong ví dụ trên.

Ví dụ phương thức POST sử dụng Form trong JSP

Chúng ta chỉnh sửa một chút ở trong ví dụ trên, để nó có thể xử lý phương thức GET cũng như POST. Chương trình JSP với tên main.jsp dưới đây sẽ xử lý input được cung cấp bởi trình duyệt web bởi sử dụng phương thức GET hoặc POST.

Sự thật là không có gì thay đổi trong JSP trên, bởi vì chỉ cách truyền các tham số được thay đổi và không có dữ liệu nhị phân nào đang được truyền tới chương trình JSP. Xử lý File liên quan tới các khái niệm sẽ được giải thích trong một chương riêng, tại đó chúng ta cần đọc các luồng dữ liệu nhị phân.

```
<html>
<head>
<title>Using GET and POST Method to Read Form Data</title>
</head>
<body>
<center>
<h1>Using GET Method to Read Form Data</h1>
<ul>
```

```

<li><p><b>First Name:</b>
    <%= request.getParameter("first_name")%>
</p></li>
<li><p><b>Last Name:</b>
    <%= request.getParameter("last_name")%>
</p></li>
</ul>
</body>
</html>

```

Sau đây là nội dung của Hello.htm file:

```

<html>
<body>
<form action="main.jsp" method="POST">
First Name: <input type="text" name="first_name">
<br />
Last Name: <input type="text" name="last_name" />
<input type="submit" value="Submit" />
</form>
</body>
</html>

```

Bây giờ, giữ main.jsp và hello.htm trong thư mục <Tomcat-installation-directory>/webapps/ROOT. Khi bạn truy cập http://localhost:8080/Hello.htm, dưới đây là output thực sự của form trên.

First Name:

First Name:

Nhập First Name và Last Name, sau đó nhấn nút Submit để xem kết quả trên thiết bị nội bộ, nơi Tomcat đang chạy.

Dựa trên input đã cung cấp, nó sẽ cho kết quả tương tự như ví dụ trên.

Truyền Checkbox Data tới chương trình JSP

Checkbox được sử dụng khi có nhiều hơn một option được yêu cầu để được chọn.

Đây là ví dụ cho HTML code, CheckBox.htm, có hai checkbox:

```

<html>
<body>
<form action="main.jsp" method="POST" target="_blank">
<input type="checkbox" name="maths" checked="checked" /> Maths
<input type="checkbox" name="physics" /> Physics
<input type="checkbox" name="chemistry" checked="checked" />
                                                    Chemistry

<input type="submit" value="Select Subject" />
</form>
</body>
</html>

```

Kết quả của code trên là form sau:

☒ Maths ☐ Physics ☒ Chemistry

Dưới đây là main.jsp để xử lý input được cung cấp bởi trình duyệt cho nút checkbox.

```

<html>
<head>
<title>Reading Checkbox Data</title>
</head>
<body>
<center>
<h1>Reading Checkbox Data</h1>
<ul>
<li><p><b>Maths Flag:</b>
        <%= request.getParameter("maths")%>
</p></li>
<li><p><b>Physics Flag:</b>
        <%= request.getParameter("physics")%>
</p></li>
<li><p><b>Chemistry Flag:</b>
        <%= request.getParameter("chemistry")%>
</p></li>
</ul>

```

```
</body>
</html>
```

Nó sẽ cho kết quả sau:

```
Reading Checkbox Data
Maths Flag : : on
Physics Flag: : null
Chemistry Flag: : on
```

Đọc tất cả Form Parameter trong JSP

Ví dụ chung sau đây sử dụng phương thức *getParameterNames()* của *HttpServletRequest* để đọc tất cả Form Parameter có sẵn. Phương thức này trả về một Enumeration (bản liệt kê) mà chứa các tên Parameter trong thứ tự chưa được sắp xếp.

Khi chúng ta có một Enumeration, lặp Enumeration này theo phương thức chuẩn: sử dụng phương thức *hasMoreElements()* để xác định khi nào cần dừng và sử dụng phương thức *nextElement()* để nhận tên của mỗi parameter.

```
<%@ page import="java.io.*,java.util.*" %>
<html>
<head>
<title>HTTP Header Request Example</title>
</head>
<body>
<center>
<h2>HTTP Header Request Example</h2>
<table width="100%" border="1" align="center">
<tr bgcolor="#949494">
<th>Param Name</th><th>Param Value(s)</th>
</tr>
<%
    Enumeration paramNames = request.getParameterNames();

    while(paramNames.hasMoreElements()) {
        String paramName = (String)paramNames.nextElement();
        out.print("<tr><td>" + paramName + "</td>\n");
        String paramValue = request.getHeader(paramName);
```

```

        out.println("<td> " + paramValue + "</td></tr>\n");
    }
%>
</table>
</center>
</body>
</html>

```

Còn đây là nội dung của Hello.htm:

```

<html>
<body>
<form action="main.jsp" method="POST" target="_blank">
<input type="checkbox" name="maths" checked="checked" /> Maths
<input type="checkbox" name="physics" /> Physics
<input type="checkbox" name="chemistry" checked="checked" /> C
hem
<input type="submit" value="Select Subject" />
</form>
</body>
</html>

```

Bây giờ, gọi JSP bởi sử dụng Hello.htm trên, nó sẽ cho kết quả sau dựa trên input đã cho:

Param Name	Param Value(s)
maths	on
chemistry	on

Bạn có thể thử JSP trên để đọc bất kỳ dữ liệu form mà có các đối tượng khác như text box, radio button hoặc dropdown box ...

Câu hỏi ôn tập.

1. Làm Thế Nào Để Truy Cập Thông Tin Được Gửi Từ Form Trong Một Trang JSP?
2. Giải thích cách sử dụng đối tượng request để truy cập dữ liệu được gửi từ một form HTML trong JSP.

3. Sự Khác Biệt Giữa Phương Thức GET và POST Trong Xử Lý Form Trong JSP là Gì?
4. Nêu rõ sự khác biệt giữa phương thức GET và POST khi xử lý form trong JSP và khi nào nên sử dụng mỗi phương thức.
5. Làm Thế Nào để Hiển Thị Dữ Liệu Được Gửi Từ Form Trên Một Trang JSP Khác?
6. Hướng dẫn cách sử dụng đối tượng request để hiển thị dữ liệu được gửi từ form trên một trang JSP khác.
7. Làm Thế Nào Để Thực Hiện Chuyển Hướng Trang Sau Khi Gửi Form Trong JSP?
8. Giải thích cách sử dụng đối tượng response để thực hiện chuyển hướng trang sau khi xử lý form trong JSP.
9. Mô tả cách sử dụng đối tượng response để xử lý lỗi và hiển thị thông báo lỗi cho người dùng trong trang JSP.

Bài 9: Lập trình JSP(Tiếp theo)

Trong bài này chúng ta sẽ tìm hiểu về thẻ chuẩn - Standard Tag Library (JSTL) trong JSP, JavaBeans và Custom Tags trong JSP cũng như Expression Language (EL) và Exception trong JSP.

4.12. Thư viện thẻ chuẩn - Standard Tag Library (JSTL) trong JSP

JavaServer Pages Standard Tag Library (JSTL) là một tập hợp các thẻ JSP hữu ích, mà gói các tính năng lõi phổ biến tới các ứng dụng JSP.

JSTL hỗ trợ tới các tác vụ phổ biến và có tính cấu trúc, ví dụ như các tính lặp và điều kiện, các thẻ để thao tác tài liệu XML, các thẻ đa ngôn ngữ, và các thẻ SQL. Nó cũng cung cấp Framework để tích hợp các Custom Tags với các thẻ JSTL.

Các thẻ JSTL có thể được phân loại, theo tính năng của nó, thành các nhóm thư viện thẻ JSTL sau, mà có thể được sử dụng khi tạo một JSP page:

- **Core Tags:** Nhóm thẻ cơ bản
- **Formatting tags:** Nhóm thẻ định dạng
- **SQL tags:** Nhóm thẻ SQL
- **XML tags:** Nhóm thẻ XML
- **JSTL Functions:** Nhóm hàm JSTL

Cài đặt thư viện JSTL

Nếu bạn đang sử dụng Apache Tomcat container, thì bạn theo hai bước sau:

- Tải bản phân phối nhị phân từ Apache Taglib và mở file nén đó.
- Để sử dụng Standard Taglib từ Jakarta Taglib, đơn giản bạn chỉ cần sao chép các JAR file trong thư mục 'lib' tới thư mục webapps\ROOT\WEB-INF\lib trong ứng dụng của bạn.

Để sử dụng bất kỳ thư viện nào, bạn phải bao một <taglib> tại trên cùng của mỗi JSP mà sử dụng thư viện đó.

Nhóm Core Tags trong JSTL

Nhóm Core Tags là các thẻ JSTL được sử dụng phổ biến nhất. Sau đây là cú pháp đơn giản để bao thư viện JSTL Core trong JSP của bạn:

```
<%@ taglib prefix="c"
      uri="http://java.sun.com/jsp/html/core" %>
```

Bảng dưới liệt kê các thẻ trong nhóm Core Tags trong JSTL:

Thẻ	Miêu tả
Thẻ <c:out > trong JSTL	Giống <%= ... >, nhưng cho các Expression
Thẻ <c:set > trong JSTL	Thiết lập kết quả của một ước lượng Expression trong một 'scope'
Thẻ <c:remove > trong JSTL	Gỡ bỏ một biến mục tiêu (từ một biến scope cụ thể, nếu đã xác định)
Thẻ <c:catch> trong JSTL	Bắt bất kỳ Throwable mà xuất hiện trong thân của nó và trưng bày nó một cách tùy ý
Thẻ <c:if> trong JSTL	Thẻ điều kiện đơn giản, mà ước lượng phần thân của nó nếu điều kiện đã cho là true
Thẻ <c:choose> trong JSTL	Thẻ điều kiện đơn giản mà thiết lập một context cho các hoạt động điều kiện loại trừ, được đánh dấu bởi <when> và <otherwise>
Thẻ <c:when> trong JSTL	Thẻ phụ của <choose> mà include phần thân của nó nếu điều kiện được ước lượng là true
Thẻ <c:otherwise > trong JSTL	Thẻ phụ của <choose> mà theo sau thẻ <when> và chỉ chạy nếu tất cả điều kiện trước được ước lượng là 'false'
Thẻ <c:import> trong JSTL	Thu nhận một URL tuyệt đối hoặc quan hệ và trưng bày nội dung của nó tới hoặc trang đó, một String

	trong 'var', hoặc một Reader trong 'varReader'
Thẻ <c:forEach > trong JSTL	Thẻ lặp cơ bản, chấp nhận nhiều kiểu tập hợp khác nhau và hỗ trợ subsetting (chia tập con) và tính năng khác
Thẻ <c:forEachTokens> trong JSTL	Lặp qua các token, được phân biệt bởi các dấu phân tách (delimiter) đã cung cấp
Thẻ <c:param> trong JSTL	Thêm một parameter tới một URL của thẻ đang chứa 'import'
Thẻ <c:redirect > trong JSTL	Redirect tới một URL mới
Thẻ <c:url> trong JSTL	Tạo một URL với các tham số truy vấn tùy ý

Bảng 4: 23 Các thẻ Core Tags trong JSTL

Nhóm Formatting Tags trong JSTL

Nhóm Formatting Tags trong JSTL được sử dụng để định dạng và hiển thị text, date, time và số ngôn ngữ trong Website. Sau đây là cú pháp cơ bản để include thư viện thẻ định dạng trong JSP của bạn.

```
<%@ taglib prefix="fmt"
      uri="http://java.sun.com/jsp/html/fmt" %>
```

Bảng dưới liệt kê các thẻ trong nhóm Formatting Tags trong JSTL:

Thẻ	Miêu tả
Thẻ <fmt:formatNumber> trong JSTL	Trả lại giá trị số với định dạng cụ thể
Thẻ <fmt:parseNumber> trong JSTL	Parse biểu diễn chuỗi của một số, tiền tệ, phần trăm
Thẻ <fmt:formatDate> trong JSTL	Định dạng một date/time bởi sử dụng Style và Pattern đã cho
Thẻ <fmt:parseDate> trong JSTL	Parse biểu diễn chuỗi của một date/time

Thẻ <fmt:bundle> trong JSTL	Tải một Resource Bundle để được sử dụng bởi phần thân thẻ
Thẻ <fmt:setLocale> trong JSTL	Lưu giữ Locale đã cho trong biến cấu hình locale
Thẻ <fmt:setBundle> trong JSTL	Tải một Resource Bundle và lưu giữ nó trong biến scope đã đặt tên hoặc biến cấu hình bundle
Thẻ <fmt:timeZone> trong JSTL	Xác định timezone cho bất kỳ định dạng time nào hoặc parse các action được lập trong phần thân của nó
Thẻ <fmt:setTimeZone> trong JSTL	Lưu giữ timezone đã cung cấp biến cấu hình time zone đó
Thẻ <fmt:message> trong JSTL	Hiển thị một thông báo đa ngôn ngữ
Thẻ <fmt:requestEncoding> trong JSTL	Thiết lập mã hóa ký tự cho request

Bảng 4: 24 Các thẻ trong nhóm Formatting Tags trong JSTL

Nhóm SQL Tags trong JSTL

Nhóm SQL Tags trong JSTL cung cấp các thẻ để tương tác với các Relational Database (RDBMSs), ví dụ như Oracle, MySQL, hoặc Microsoft SQL Server.

Đây là cú pháp để bao nhóm SQL Tags trong JSTL trong JSP của bạn:

```
<%@ taglib prefix="sql"
    uri="http://java.sun.com/jsp/html/sql" %>
```

Bảng dưới liệt kê các thẻ trong nhóm SQL Tags trong JSP:

Thẻ	Miêu tả
Thẻ <sql:setDataSource> trong JSTL	Tạo một DataSource đơn giản chỉ thích hợp cho prototype
Thẻ <sql:query> trong JSTL	Thực thi SQL query được định nghĩa trong phần thân của nó hoặc thông qua thuộc tính sql

Thẻ <sql:update> trong JSTL	Thực thi SQL update được định nghĩa trong phần thân của nó hoặc thông quan thuộc tính sql
Thẻ <sql:param> trong JSTL	Thiết lập một parameter trong một lệnh SQL tới giá trị đã xác định
Thẻ <sql:dateParam> trong JSTL	Thiết lập một parameter trong một lệnh SQL tới giá trị java.util.Date đã xác định
Thẻ <sql:transaction > trong JSTL	Cung cấp các phần tử database action được lập với một Connection đã chia sẻ, thiết lập để thực thi tất cả các lệnh

Bảng 4: 25 Các thẻ SQL Tags trong JSP

Nhóm XML Tags trong JSTL

Nhóm XML Tags trong JSTL cung cấp một cách để tạo và thao tác các tài liệu XML. Sau đây là cú pháp để include nhóm XML Tags trong JSP của bạn:

Thư viện thẻ XML Tags trong JSTL có các custom tags để tương tác với XML data. Điều này bao gồm việc parse XML, truyền tải XML data, và điều khiển dòng dữ liệu dựa trên các XPath Expression.

```
<%@ taglib prefix="x"
      uri="http://java.sun.com/jsp/html/xml" %>
```

Trước khi đi vào ví dụ, bạn cần copy hai XML sau và các thư viện XPath liên quan vào trong <Tomcat Installation Directory>\lib:

- **XercesImpl.jar:** Tải nó từ <http://www.apache.org/dist/xerces/j/>
- **xalan.jar:** Tải nó từ <http://xml.apache.org/xalan-j/index.html>

Dưới đây liệt kê các thẻ trong nhóm XML Tags trong JSTL:

Thẻ	Miêu tả
Thẻ <x:out> trong JSTL	Giống <%= ... >, nhưng dành cho các XPath Expression
Thẻ <x:parse> trong JSTL	Sử dụng để <i>parse</i> XML data được xác định hoặc thông qua một thuộc tính hoặc trong phần thân thẻ
Thẻ <x:set > trong JSTL	Thiết lập một biến tới giá trị của một XPath expression

Thẻ <x:if > trong JSTL	Ước lượng XPath Expression và nếu nó là true, thì nó xử lý phần thân thẻ. Nếu điều kiện là false, phần thân bị bỏ qua
Thẻ <x:forEach> trong JSTL	Lặp qua các node trong một tài liệu XML
Thẻ <x:choose> trong JSTL	Thẻ điều kiện đơn giản mà thiết lập một context cho hoạt động điều kiện loại trừ, được đánh dấu bởi <when> và <otherwise> trong JSTL
Thẻ <x:when > trong JSTL	Thẻ phụ của <choose> mà bao phần thân thẻ nếu Expression được ước lượng là 'true'
Thẻ <x:otherwise > trong JSTL	Thẻ phụ của <choose> mà theo sau các thẻ <when> và chỉ chạy khi nếu tất cả điều kiện trước là 'false'
Thẻ <x:transform > trong JSTL	Áp dụng một phép biến đổi XSL trên một tài liệu XML
Thẻ <x:param > trong JSTL	Sử dụng cùng với thẻ transform để thiết lập một parameter trong XSLT stylesheet

Bảng 4: 26 Các thẻ trong nhóm XML Tags trong JSTL

Nhóm JSTL Functions

JSTL bao gồm một số hàm chuẩn, mà hầu hết là các hàm thao tác chuỗi phổ biến. Sau đây là cú pháp để bao nhóm JSTL Functions trong chương trình JSP của bạn:

```
<%@ taglib prefix="fn"
    uri="http://java.sun.com/jsp/html/functions" %>
```

Bảng dưới liệt kê danh sách các JSTL Functions:

Hàm	Miêu tả
Hàm fn:contains() trong JSTL	Kiểm tra nếu một chuỗi input chứa chuỗi phụ đã cho
Hàm fn:containsIgnoreCase() trong JSTL	Kiểm tra nếu một chuỗi input chứa chuỗi phụ đã cho trong trường hợp không phân biệt kiểu chữ

Hàm fn:endsWith() trong JSTL	Kiểm tra nếu một chuỗi input kết thúc với suffix đã cho
Hàm fn:escapeXml() trong JSTL	Các ký tự thoát mà có thể được phiên dịch như XML markup
Hàm fn:indexOf() trong JSTL	Trả về index bên trong một chuỗi về sự xuất hiện đầu tiên của chuỗi phụ
Hàm fn:join() trong JSTL	Kết hợp tất cả phần tử trong một mảng thành một chuỗi
Hàm fn:length() trong JSTL	Trả về số item trong một tập hợp, hoặc số ký tự trong một chuỗi
Hàm fn:replace() trong JSTL	Trả về một chuỗi là kết quả của việc thay thế một chuỗi input với một chuỗi đã cho
Hàm fn:split() trong JSTL	Chia một chuỗi thành một mảng các chuỗi phụ
Hàm fn:startsWith() trong JSTL	Kiểm tra nếu một chuỗi input bắt đầu với prefix đã cho
Hàm fn:substring() trong JSTL	Trả về một tập con của một chuỗi
Hàm fn:substringAfter() trong JSTL	Trả về một tập con của một chuỗi ở sau một chuỗi phụ đã cho
Hàm fn:substringBefore() trong JSTL	Trả về một tập con của một chuỗi ở trước một chuỗi phụ đã cho
Hàm fn:toLowerCase() trong JSTL	Biến đổi tất cả ký tự của một chuỗi thành chữ thường
Hàm fn:toUpperCase() trong JSTL	Biến đổi tất cả ký tự của một chuỗi thành chữ hoa

Hàm fn:trim() trong JSTL	Gỡ bỏ các khoảng trống trắng từ hai đầu của một chuỗi
--------------------------	---

Bảng 4: 27 Các JSTL Functions

4.13. JavaBeans trong JSP

Một JavaBean là một lớp Java được xây dựng một cách đặc biệt trong Java và được mã hóa theo JavaBeans API Specifications.

Sau đây là các đặc trưng duy nhất giúp phân biệt một JavaBean với các lớp khác trong Java:

- Nó cung cấp một constructor mặc định, không có tham số.
- Nó có thể xếp thứ tự và triển khai **Serializable** interface.
- Nó có thể có một số đặc tính mà có thể được đọc và được viết.
- Nó có thể có một số phương thức "getter" và "setter" cho các đặc tính.

Các đặc tính của JavaBeans

Một đặc tính của JavaBean là một thuộc tính được đặt tên mà có thể được truy cập bởi người sử dụng đối tượng đó. Thuộc tính này có thể là bất kỳ kiểu dữ liệu Java nào, bao gồm cả các lớp mà bạn định nghĩa.

Một đặc tính của JavaBean có thể là read, write, read-only, hoặc write-only. Các đặc tính JavaBeans được truy cập thông qua hai phương thức trong lớp triển khai của JavaBean:

Phương thức	Miêu tả
<code>getPropertyName()</code>	Ví dụ, nếu tên đặc tính là <i>firstName</i> , thì tên phương thức sẽ là <code>getFirstName()</code> để đọc đặc tính đó. Phương thức này được gọi bởi accessor
<code>setPropertyName()</code>	Ví dụ, nếu tên đặc tính là <i>firstName</i> , thì tên phương thức sẽ là <code>setFirstName()</code> để viết phương thức đó. Phương thức này được gọi bởi mutator.

Bảng 4: 28 Các phương thức truy nhập thuộc tính của JavaBean

Một thuộc tính read-only sẽ chỉ có một phương thức `getPropertyName()`, và một thuộc tính write-only sẽ chỉ có một phương thức `setPropertyName()`.

Ví dụ về JavaBeans

Bạn xem xét một Student class với một số đặc tính sau:

```
package com.tutorialspoint;
```

```

public class StudentsBean implements java.io.Serializable
{
    private String firstName = null;
    private String lastName = null;
    private int age = 0;
    public StudentsBean() {
    }
    public String getFirstName(){
        return firstName;
    }
    public String getLastName(){
        return lastName;
    }
    public int getAge(){
        return age;
    }
    public void setFirstName(String firstName){
        this.firstName = firstName;
    }
    public void setLastName(String lastName){
        this.lastName = lastName;
    }
    public void setAge(Integer age){
        this.age = age;
    }
}

```

Truy cập JavaBeans

useBean action khai báo một JavaBean để sử dụng trong một JSP. Một khi đã được khai báo, Bean này trở thành một biến scripting mà có thể được truy cập bởi cả các phân tử scripting và các Custom Tags khác (custom tag) được sử dụng trong JSP. Cú pháp đầy đủ để sử dụng thẻ useBean là:

```
<jsp:useBean id="bean's name" scope="bean's scope" typeSpec/>
```

Ở đây, các giá trị cho thuộc tính scope có thể là page, request, session hoặc application tùy theo yêu cầu của bạn. Giá trị của thuộc tính id có thể là bất kỳ giá trị nào, miễn là nó là một tên duy nhất giữa các khai báo useBean trong cùng JSP.

Ví dụ sau minh họa cách sử dụng đơn giản của nó:

```
<html>
<head>
<title>useBean Example</title>
</head>
<body>
<jsp:useBean id="date" class="java.util.Date" />
<p>The date/time is <%= date %>
</body>
</html>
```

Nó sẽ cho kết quả sau:

```
The date/time is Thu Sep 30 11:18:11 GST 2010
```

Truy cập các Property của JavaBeans

Cùng với <jsp:useBean...>, bạn có thể sử dụng <jsp:getProperty/> để truy cập các phương thức get và <jsp:setProperty/> action để truy cập các phương thức set. Đây là cú pháp đầy đủ:

```
<jsp:useBean id="id" class="bean's class" scope="bean's scope"
>
    <jsp:setProperty name="bean's id" property="property name"
                    value="value"/>
    <jsp:getProperty name="bean's id" property="property name"/
>
    .....
</jsp:useBean>
```

Thuộc tính tham chiếu name id của một JavaBean được giới thiệu trước đó tới JSP bởi useBean action. Thuộc tính property là tên của phương thức get hoặc set nên được gọi.

Ví dụ đơn giản sau để truy cập dữ liệu bởi sử dụng cú pháp trên trong JSP:

```
<html>
<head>
<title>get and set properties Example</title>
```

```
</head>
<body>
<jsp:useBean id="students"
              class="com.tutorialspoint.StudentsBean">
    <jsp:setProperty name="students" property="firstName"
                    value="Zara"/>
    <jsp:setProperty name="students" property="lastName"
                    value="Ali"/>
    <jsp:setProperty name="students" property="age"
                    value="10"/>
</jsp:useBean>
<p>Student First Name:
    <jsp:getProperty name="students" property="firstName"/>
</p>
<p>Student Last Name:
    <jsp:getProperty name="students" property="lastName"/>
</p>
<p>Student Age:
    <jsp:getProperty name="students" property="age"/>
</p>
</body>
</html>
```

Tạo StudentsBean.class có sẵn trong CLASSPATH và thử truy cập JSP trên. Nó sẽ cho kết quả sau:

```
Student First Name: Zara

Student Last Name: Ali

Student Age: 10
```

4.14. Custom Tags trong JSP

Một custom tag là một phần tử JSP được định nghĩa bởi người sử dụng. Khi một JSP page chứa một custom tag được biên dịch thành một Servlet, thẻ này được biến đổi để hoạt động trên một đối tượng đã gọi một Tag Handler. Sau đó, Web container triệu hồi các hoạt động này khi Servlet của JSP page được thực thi.

Sự kế thừa của các JSP tag giúp bạn tạo các thẻ mới mà bạn có thể chèn một cách trực tiếp vào trong một JSP page, như khi bạn thực hiện với các thẻ có sẵn (built-in) trong các chương trước đã giới thiệu. JSP 2.0 Specification giới thiệu Simple Tag Handler để viết các Custom Tag này.

Để viết một custom tag, đơn giản bạn có thể kế thừa lớp SimpleTagSupport và ghi đè phương thức doTag(), tại đây bạn có thể đặt code của bạn để tạo content cho thẻ đó.

Tạo thẻ "Hello" trong JSP

Giả sử bạn muốn định nghĩa một custom tag tên là <ex:Hello> và bạn muốn sử dụng nó trong kiểu mà không có phần thân như sau:

```
<ex:Hello />
```

Để tạo một Custom Tag trong JSP, đầu tiên bạn phải tạo một lớp Java mà hoạt động như là một Tag Handler. Vì thế, chúng ta tạo HelloTag class như sau:

```
package com.tutorialspoint;

import javax.servlet.jsp.tagext.*;
import javax.servlet.jsp.*;
import java.io.*;

public class HelloTag extends SimpleTagSupport {
    public void doTag() throws JspException, IOException {
        JspWriter out = getJspContext().getOut();
        out.println("Hello Custom Tag!");
    }
}
```

Code trên có mã hóa đơn giản, với phương thức doTag() nhận đối tượng JspContext hiện tại bởi sử dụng phương thức getJspContext() và sử dụng nó để gửi "Hello Custom Tag!" tới đối tượng JspWriter hiện tại.

Biên dịch lớp trên và copy nó trong một thư mục có sẵn trong biến môi trường CLASSPATH. Cuối cùng, tạo file thư viện thẻ sau: <Tomcat-Installation-Directory>webapps\ROOT\WEB-INF\custom.tld.

```
<taglib>
  <tlib-version>1.0</tlib-version>
  <jsp-version>2.0</jsp-version>
  <short-name>Example TLD</short-name>
  <tag>
```

```

    <name>Hello</name>
    <tag-class>com.tutorialspoint.HelloTag</tag-class>
    <body-content>empty</body-content>
</tag>
</taglib>

```

Bây giờ là lúc sử dụng custom tag là Hello đã định nghĩa trên trong chương trình JSP như sau:

```

<%@ taglib prefix="ex" uri="WEB-INF/custom.tld"%>
<html>
    <head>
        <title>A sample custom tag</title>
    </head>
    <body>
        <ex:Hello/>
    </body>
</html>

```

Thử gọi JSP trên và nó sẽ cho kết quả sau:

```
Hello Custom Tag!
```

Truy cập phần thân thẻ trong JSP

Bạn có thể bao một message trong phần thân của Custom Tags như khi bạn đã thấy với các thẻ chuẩn. Giả sử bạn muốn định nghĩa một custom tag với tên `<ex:Hello>` và bạn muốn sử dụng nó trong kiểu có một phần thân thẻ:

```

<ex:Hello>
    This is message body
</ex:Hello>

```

Chúng ta sửa đổi code trên một chút để xử lý phần thân cho thẻ:

```

package com.tutorialspoint;
import javax.servlet.jsp.tagext.*;
import javax.servlet.jsp.*;
import java.io.*;
public class HelloTag extends SimpleTagSupport {
    StringWriter sw = new StringWriter();
    public void doTag()

```

```

        throws JspException, IOException
    {
        getJspBody().invoke(sw);
        getJspContext().getOut().println(sw.toString());
    }
}

```

Trong trường hợp này, kết quả từ sự triệu hồi, đầu tiên được bắt vào trong một `StringWriter` trước khi được ghi tới `JspWriter` mà liên kết với thẻ đó. Bây giờ, theo đó, chúng ta cần thay đổi TLD file như sau:

```

<taglib>
  <tlib-version>1.0</tlib-version>
  <jsp-version>2.0</jsp-version>
  <short-name>Example TLD with Body</short-name>
  <tag>
    <name>Hello</name>
    <tag-class>com.tutorialspoint.HelloTag</tag-class>
    <body-content>scriptless</body-content>
  </tag>
</taglib>

```

Chúng ta gọi thẻ trên với phần thân chính xác như sau:

```

<%@ taglib prefix="ex" uri="WEB-INF/custom.tld"%>
<html>
  <head>
    <title>A sample custom tag</title>
  </head>
  <body>
    <ex:Hello>
      This is message body
    </ex:Hello>
  </body>
</html>

```

Nó sẽ cho kết quả:

```
This is message body
```

Thuộc tính của Custom Tag trong JSP

Ta có thể sử dụng các thuộc tính đa dạng cùng với các thẻ custom tag. Để chấp nhận một giá trị thuộc tính, một lớp custom tag cần triển khai các phương thức setter, như phương thức setter trong JavaBeans như sau:

```
package com.tutorialspoint;
import javax.servlet.jsp.tagext.*;
import javax.servlet.jsp.*;
import java.io.*;

public class HelloTag extends SimpleTagSupport {
    private String message;
    public void setMessage(String msg) {
        this.message = msg;
    }
    StringWriter sw = new StringWriter();
    public void doTag()
        throws JspException, IOException
    {
        if (message != null) {
            /* Use message from attribute */
            JspWriter out = getJspContext().getOut();
            out.println( message );
        }
        else {
            /* use message from the body */
            getJspBody().invoke(sw);
            getJspContext().getOut().println(sw.toString());
        }
    }
}
```

Tên của thuộc tính là "message", vì thẻ phương thức setter là setMessage(). Bây giờ chúng ta thêm thuộc tính này trong TLD file bởi sử dụng phần tử <attribute> như sau:

```
<taglib>
```

```

<tlib-version>1.0</tlib-version>
<jsp-version>2.0</jsp-version>
<short-name>Example TLD with Body</short-name>
<tag>
  <name>Hello</name>
  <tag-class>com.tutorialspoint.HelloTag</tag-class>
  <body-content>scriptless</body-content>
  <attribute>

```

```

    <name>message</name>
  </attribute>
</tag>
</taglib>

```

Thử JSP trên với thuộc tính message như sau:

```

<%@ taglib prefix="ex" uri="WEB-INF/custom.tld"%>
<html>
  <head>
    <title>A sample custom tag</title>
  </head>
  <body>
    <ex:Hello message="This is custom tag" />
  </body>
</html>

```

Nó sẽ cho kết quả sau:

This is custom tag

Property	Mục đích
name	Phần tử name định nghĩa tên của một thuộc tính. Mỗi tên thuộc tính phải là duy nhất cho một thẻ cụ thể
required	Xác định nếu thuộc tính này được yêu cầu hoặc là tùy ý. Nó

	sẽ là false cho tùy ý
rtexprvalue	Khai báo nếu một giá trị runtime expression cho một thuộc tính thẻ là hợp lệ
type	Định nghĩa kiểu lớp Java cho thuộc tính này. Theo mặc định, nó được xem như là String
Miêu tả	Sự miêu tả có tính thông tin này có thể được cung cấp
fragment	Khai báo nếu giá trị thuộc tính này nên được đối xử như là một JspFragment .

Bảng 4: 29 Các thuộc tính của Custom Tag trong JSP

Ví dụ sau xác định các Property liên quan tới một thuộc tính trong:

```
.....
<attribute>
  <name>attribute_name</name>
  <required>false</required>
  <type>java.util.Date</type>
  <fragment>false</fragment>
</attribute>
.....
```

Nếu bạn đang sử dụng hai thuộc tính, thì bạn có thể sửa đổi TLD như sau:

```
.....
<attribute>
  <name>attribute_name1</name>
  <required>false</required>
  <type>java.util.Boolean</type>
  <fragment>false</fragment>
</attribute>
<attribute>
  <name>attribute_name2</name>
  <required>true</required>
  <type>java.util.Date</type>
```

```
</attribute>
```

```
.....
```

4.15. Expression Language (EL) trong JSP

JSP Expression Language (EL) giúp dễ dàng truy cập dữ liệu ứng dụng được lưu giữ trong các thành phần JavaBeans. JSP EL cho phép bạn tạo các Expression, gồm số học và logic. Bên trong một JSP EL, bạn có thể sử dụng các integer, các số floating point, string, các hằng có sẵn true hoặc false cho các giá trị Boolean, và null.

4.15.1. Cú pháp đơn giản cho JSP EL

Đặc trưng, khi bạn xác định một giá trị thuộc tính trong một thẻ JSP, một cách đơn giản, bạn sử dụng một string. Ví dụ:

```
<jsp:setProperty name="box" property="perimeter" value="100"/>
```

JSP EL cho phép bạn xác định một Expression cho bất kỳ giá trị thuộc tính nào. Một cú pháp đơn giản cho JSP EL như sau:

Ở đây, expr xác định chính Expression. Các toán tử phổ biến nhất trong JSP EL là . và []. Hai toán tử này cho phép bạn truy cập các thuộc tính đa dạng của JavaBeans và các đối tượng JSP có sẵn.

Ví dụ, cú pháp thẻ <jsp:setProperty> trên có thể được viết với một Expression giống như:

```
<jsp:setProperty name="box" property="perimeter"  
value="0"/>
```

Khi JSP compiler thấy \${} form trong một thuộc tính, nó tạo code để ước lượng expression này và thay thế giá trị của expression đó.

Bạn cũng có thể sử dụng các JSP EL bên trong Template Text cho một thẻ. Ví dụ, thẻ <jsp:text> chèn nội dung của nó bên trong phần thân của một JSP. Khai báo <jsp:text> sau chèn <h1>Hello JSP!</h1> vào trong JSP output:

```
<jsp:text>  
<h1>Hello JSP!</h1>  
</jsp:text>
```

Bạn có thể bao một JSP EL trong phần thân của một thẻ <jsp:text> hoặc bất kỳ thẻ khác) với cùng cú pháp \${} bạn sử dụng cho các thuộc tính. Ví dụ:

```
<jsp:text>
Box Perimeter is: 0
</jsp:text>
```

EL expression có thể sử dụng các dấu ngoặc đơn để nhóm các subexpression. Ví dụ, 9 bằng 9, nhưng 7 bằng 7.

Để vô hiệu hóa sự ước lượng của các EL expression, chúng ta xác định thuộc tính **isELIgnored** của page Directive như sau:

```
<%@ page isELIgnored ="true|false" %>
```

Các giá trị hợp lệ của thuộc tính này là true hoặc false. Nếu nó là true, EL expression bị bỏ qua khi chúng xuất hiện trong các thuộc tính thẻ hoặc static text. Nếu nó là false, các EL expression được ước lượng bởi Container.

4.15.2. Toán tử cơ bản trong EL

JSP Expression Language (EL) hỗ trợ hầu hết các toán tử số học và logic được hỗ trợ bởi Java. Dưới đây là danh sách các toán tử được sử dụng thường xuyên nhất.

Toán tử	Miêu tả
.	Truy cập một đặc tính của Bean hoặc Map entry
[]	Truy cập một phân tử mảng hoặc List
()	Nhóm một subexpression để thay đổi thứ tự ước lượng
+	Phép cộng
-	Phép trừ hoặc phủ định một giá trị
*	Phép nhân
/ hoặc div	Phép chia
% hoặc mod	Phép chia lấy phần dư
== hoặc eq	Kiểm tra có bằng hay không

!= hoặc ne	Kiểm tra tính không bằng
< hoặc lt	Kiểm tra tính nhỏ hơn
> hoặc gt	Kiểm tra tính lớn hơn
<= hoặc le	Kiểm tra tính nhỏ hơn hoặc bằng
>= hoặc ge	Kiểm tra tính lớn hơn hoặc bằng
&& hoặc and	Phép AND logic
hoặc or	Phép OR logic
! hoặc not	Phản bù Boolean một ngôi
empty	Kiểm tra các giá trị biến rỗng

Bảng 4: 30 Các toán tử cơ bản trong EL

4.15.3. Hàm trong JSP EL

JSP EL cũng cho phép bạn sử dụng các hàm trong Expression. Những hàm này phải được định nghĩa trong các thư viện custom tag. Một sự sử dụng hàm có cú pháp sau:

```
${ns:func(param1, param2, ...)}
```

Tại đây, ns là không gian tên của hàm đó, func là tên hàm và param1 là giá trị tham số đầu tiên. Ví dụ, hàm fn:length, mà là một phần của thư viện JSTL có thể được sử dụng như sau để nhận độ dài của chuỗi.

```
${fn:length("Get my length")}
```

Để sử dụng một hàm từ bất kỳ thư viện thẻ nào (Standard hoặc Custom), bạn phải cài đặt thư viện đó trên Server và phải bao thư viện đó trong JSP bởi sử dụng <taglib> directive như đã giải thích trong chương JSTL

Các đối tượng ẩn trong JSP EL

JSP Expression Language hỗ trợ các đối tượng ẩn sau:

Đối tượng ẩn	Miêu tả
pageScope	Các biến scope từ phạm vi page
requestScope	Các biến scope từ phạm vi request
sessionScope	Các biến scope từ phạm vi session
applicationScope	Các biến scope từ phạm vi application
param	Các tham số request, dạng các chuỗi
paramValues	Các tham số request, dạng các tập hợp của các chuỗi
header	HTTP request header dạng các chuỗi
headerValues	HTTP request header dạng các tập hợp của các chuỗi
initParam	Các tham số khởi tạo context
cookie	Các giá trị Cookie
pageContext	Đối tượng JSP PageContext cho page hiện tại

Bảng 4: 31 Các đối tượng ẩn của EL

Bạn có thể sử dụng các đối tượng này trong một Expression khi nếu chúng là các biến. Dưới đây là một số ví dụ mà giải thích rõ ràng khái niệm trên:

Đối tượng pageContext trong JSP

Thông qua đối tượng pageContext, bạn có thể truy cập tới đối tượng request. Ví dụ, để truy cập chuỗi truy vấn đang đến cho một request, bạn có thể sử dụng Expression:

Đối tượng Scope trong JSP

Các biến `pageScope`, `requestScope`, `sessionScope`, và `applicationScope` cung cấp sự truy cập tới các biến được lưu giữ tại mỗi mức độ scope.

Ví dụ, nếu bạn cần truy cập một cách rõ ràng tới biến `box` trong phạm vi `application`, bạn có thể truy cập nó thông qua biến `applicationScope` dạng `applicationScope.box`.

Đối tượng param và paramValues trong JSP

Các đối tượng `param` và `paramValues` giúp bạn truy cập tới các giá trị tham số một cách thông thường thông qua các phương thức

`request.getParameter` và `request.getParameterValues`.

Ví dụ, để truy cập thứ tự các tham số đã đặt tên, sử dụng expression là: hoặc .

Ví dụ sau truy cập một tham số request với tên `username`:

```
<%@ page import="java.io.*,java.util.*" %>
<%
    String title = "Accessing Request Param";
%>
<html>
<head>
<title><% out.print(title); %></title>
</head>
<body>
<center>
<h1><% out.print(title); %></h1>
</center>
<div align="center">
<p></p>
</div>
</body>
</html>
```

Đối tượng `param` trả về các giá trị chuỗi đơn, trong khi đó đối tượng `paramValues` trả về các mảng chuỗi.

Đối tượng header và headerValues trong JSP

Đối tượng header và headerValues giúp bạn truy cập tới các giá trị header có sẵn một cách thông thường thông qua các phương thức

request.getHeader và request.getHeaders.

Ví dụ, để truy cập một header với tên user-agent, sử dụng expression là: 0 hoặc Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/73.0.3683.103 Safari/537.36.

Ví dụ sau truy cập một tham số header với tên user-agent:

```
<%@ page import="java.io.*,java.util.*" %>
<%
    String title = "User Agent Example";
%>
<html>
<head>
<title><% out.print(title); %></title>
</head>
<body>
<center>
<h1><% out.print(title); %></h1>
</center>
<div align="center">
<p>Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/73.0.3683.103 Safari/537.36</p>
</div>
</body>
</html>
```

Nó sẽ cho kết quả giống như:

User Agent Example

Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64; Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR 3.0.30729; Media Center PC 6.0; HPNTDF; .NET4.0C; InfoPath.2)

Đối tượng header trả về các giá trị chuỗi đơn, trong khi đối tượng headerValues trả về các mảng chuỗi.

Câu hỏi ôn tập.

1. Làm Thế Nào JSTL giúp giảm lặp lại mã trong JSP?. Mô tả cách JSTL (Standard Tag Library) giúp giảm lặp lại mã trong trang JSP và cung cấp các thẻ quan trọng.
2. JavaBeans trong JSP là gì và làm thế nào sử dụng chúng?. Giải thích vai trò của JavaBeans trong JSP và cách sử dụng chúng để truy cập và hiển thị dữ liệu.
3. Custom Tags trong jsp làm thế nào để tạo ra và sử dụng?. Hướng dẫn cách tạo và sử dụng custom tags trong JSP để tái sử dụng mã và tạo các thành phần tùy chỉnh.
4. Định nghĩa Expression Language (EL) trong JSP và cách sử dụng nó để đơn giản hóa cú pháp và truy cập dữ liệu.

Bài 10: Lập trình JSP (Tiếp theo)

Trong phần này chúng ta sẽ ôn tập và tìm hiểu các khái niệm liên quan đến xử lý ngoại lệ và bảo mật trong JSP.

4.16. Xử lý ngoại lệ (Exception) trong JSP

Khi viết JSP code, lập trình viên có thể tạo lỗi ở bất cứ phần nào của code. Bạn có thể có các loại Error sau trong JSP code.

- **Checked exceptions:** Một Checked exception là một lỗi người sử dụng đặc trưng hoặc một vấn đề mà không thể dự báo trước bởi lập trình viên. Ví dụ, nếu một file cần được mở, nhưng nó file đó là không thể tìm thấy, thì một exception xuất hiện. Những exception này không thể bị bỏ qua một cách đơn giản tại thời điểm biên dịch.
- **Runtime exceptions:** Một Runtime exception là một ngoại lệ mà xảy ra có thể được tránh bởi lập trình viên. Trái ngược với Checked Exception, runtime exception được bỏ qua tại thời điểm biên dịch.
- **Errors:** Chúng không hoàn toàn là exception, nhưng là các vấn đề mà xuất hiện vượt quá sự điều khiển của người dùng hoặc lập trình viên. Nét đặc trưng là, các Error được bỏ qua trong code bởi vì bạn có thể hiếm khi thực hiện điều gì đó về một Error này. Ví dụ, một sự tràn stack (stack overflow) xuất hiện, một error sẽ xuất hiện Chúng cũng được bỏ qua tại thời điểm biên dịch.

Chương này cung cấp cho bạn các cách đơn giản và hiệu quả để xử lý runtime exception/error xảy ra trong JSP code.

Sử dụng đối tượng Exception trong JSP

Đối tượng exception là một trường hợp của lớp phụ của Throwable (ví dụ: java.lang. NullPointerException) và chỉ có sẵn trong các error page. Sau đây là danh sách các phương thức quan trọng trong lớp Throwable.

STT	Phương thức và Miêu tả
1	public String getMessage(): Trả về một thông báo chi tiết về Exception đã xuất hiện đó. Thông báo này được khởi tạo trong Throwable constructor
2	public Throwable getCause(): Trả về nguyên nhân của Exception khi được biểu diễn bởi một đối tượng Throwable
3	public String toString(): Trả về tên của lớp được nối chuỗi với kết quả của getMessage()
4	public void printStackTrace(): In kết quả của toString() cùng với stack trace tới System.err (Error Output Stream)
5	public StackTraceElement [] getStackTrace(): Trả về một mảng chứa mỗi phần tử trên Stack trace. Phần tử tại chỉ mục 0 biểu diễn phần trên cùng của stack đang gọi, và phần tử cuối cùng trong mảng biểu diễn phương thức đó tại cuối của stack đang gọi
6	public Throwable fillInStackTrace(): Điền vào stack trace của đối tượng Throwable này với stack trace hiện tại, thêm tới bất kỳ phân thông tin trước đó trong stack trace

Bảng 4: 32 Các phương thức quan trọng trong lớp Throwable

JSP cung cấp cho bạn một tùy chọn để xác định Error Page cho mỗi JSP. Bất cứ khi nào page ném một exception, JSP container tự động gọi error page.

Sau đây là một ví dụ đơn giản để xác định một error page cho một main.jsp. Để thiết lập một error page, sử dụng `<%@ page errorPage="xxx" %>` directive.

```

<%@ page errorPage="ShowError.jsp" %>

<html>
<head>
    <title>Error Handling Example</title>
</head>
<body>
<%
    // Throw an exception to invoke the error page
    int x = 1;
    if (x == 1)
    {
        throw new RuntimeException("Error condition!!!");
    }

```

```

    }
%>
</body>
</html>

```

Bây giờ bạn sẽ phải viết một chương trình Xử lý lỗi trong JSP là ShowError.jsp, được cung cấp dưới đây. Chú ý rằng, trang xử lý lỗi bao gồm directive là `<%@ page isErrorPage="true" %>`. Directive này làm JSP compiler để tạo biến exception instance.

```

<%@ page isErrorPage="true" %>
<html>
<head>
<title>Show Error Page</title>
</head>
<body>
<h1>Opps...</h1>
<p>Sorry, an error occurred.</p>
<p>Here is the exception stack trace: </p>
<pre>
<% exception.printStackTrace(response.getWriter()); %>
</pre>
</body>
</html>

```

Thử truy cập main.jsp trên, nó sẽ tạo ra:

```

java.lang.RuntimeException: Error condition!!!
.....

Opps...
Sorry, an error occurred.

Here is the exception stack trace:

```

Sử dụng JSTL tags cho Error Page trong JSP

Bạn có thể sử dụng JSTL tags để viết một error page là ShowError.jsp. Trang này có logic giống như những gì chúng ta đã sử dụng trong ví dụ trên, nhưng có cấu trúc tốt hơn và nó cung cấp nhiều thông tin hơn:

```

<%@ taglib prefix="c" uri="http://java.sun.com../jsp/jstl/core
" %>

```

```
<%@page isErrorPage="true" %>
<html>
<head>
<title>Show Error Page</title>
</head>
<body>
<h1>Oops...</h1>
<table width="100%" border="1">
<tr valign="top">
<td width="40%"><b>Error:</b></td>
<td></td>
</tr>
<tr valign="top">
<td><b>URI:</b></td>
<td></td>
</tr>
<tr valign="top">
<td><b>Status code:</b></td>
<td>0</td>
</tr>
<tr valign="top">
<td><b>Stack trace:</b></td>
<td>
<c:forEach var="trace"
            items="">
<p></p>
</c:forEach>
</td>
</tr>
</table>
</body>
</html>
```

Thử truy cập main.jsp, nó sẽ cho kết quả:

```
Error:                java.lang.RuntimeException: Error condition!!!
URI:                  /main.jsp
Status code:          500
Stack trace:          org.apache.jsp.main_jsp._jspService(main_jsp.java:65)
                     org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:68)
                     javax.servlet.http.HttpServlet.service(HttpServlet.java:722)
                     org.apache.jasper.servlet.JspServlet.service(JspServlet.java:265)
                     javax.servlet.http.HttpServlet.service(HttpServlet.java:722)
                     .....

```

Sử dụng khối Try...Catch trong JSP

Nếu bạn muốn xử lý các Error bên trong cùng một trang và muốn thực hiện một số hành động thay vì kích hoạt một error page, bạn có thể sử dụng khối try...catch trong JSP.

Sau đây là ví dụ đơn giản minh họa cách sử dụng khối try...catch trong JSP. Đặt code trên trong main.jsp:

```
<html>
<head>
    <title>Try...Catch Example</title>
</head>
<body>
<%
    try{
        int i = 1;
        i = i / 0;
        out.println("The answer is " + i);
    }
    catch (Exception e){
        out.println("An exception occurred: " + e.getMessage());
    }
%>
</body>

```

```
</html>
```

Thử truy cập main.jsp, nó sẽ cho kết quả sau:

```
An exception occurred: / by zero
```

4.17. Debug trong JSP

Công việc test/debug một JSP vfa Servlet luôn luôn là khó khăn và phức tạp. JSP và Servlet bao gồm một lượng tương tác Server-Client lớn, do đó, việc tạo ra lỗi là không thể không tránh khỏi.

Dưới đây là một số gợi ý có thể sẽ giúp ích trong việc debug chương trình JSP hoặc Servlet.

Phương thức System.out.println()

Phương thức System.out.println() là dễ để sử dụng như là một Marker để kiểm tra có hay không một phần nào đó của code đang được thực thi hoặc không. Chúng ta cũng có thể in các giá trị của biến. Ngoài ra:

Vì đối tượng System là một phần của các đối tượng Core Java, nó có thể được sử dụng bất cứ đâu mà không cần cài đặt bất kỳ lớp bổ sung nào. Nó bao gồm Servlet, JSP, RMI, EJB's, ordinary Beans, các class, và các standalone application.

So sánh với việc dừng tại các Breakpoint, việc write tới System.out không gây trở ngại nhiều tới luồng thực thi thông thường của ứng dụng, mà làm cho nó đáng quý.

Sau đây là cú pháp của System.out.println():

```
System.out.println("Debugging message");
```

Dưới đây là ví dụ đơn giản sử dụng phương thức System.out.println():

```
%>
<%@taglib prefix="c" uri="http://java.sun.com../jsp/jstl/core"
<html>
<head><title>System.out.println</title></head>
<body>
<c:forEach var="counter" begin="1" end="10" step="1" >
    <c:out value="-5"/></br>
    <% System.out.println( "counter= " +
                                pageContext.findAttribute("counter") ); %
>
</c:forEach>
</body>
</html>
```

Bây giờ nếu truy cập JSP trên, nó sẽ cho kết quả sau:

```
-4
-3
-2
-1
0
1
2
3
4
5
```

Nếu sử dụng Tomcat, ta cũng sẽ tìm thấy các dòng này được phụ thêm tới phần cuối của stdout.log trong thư mục logs.

```
counter=1
counter=2
counter=3
counter=4
counter=5
counter=6
counter=7
counter=8
counter=9
counter=10
```

Với cách này, bạn có thể in các biến và thông tin khác vào trong system log, mà có thể được phân tích để tìm nguyên nhân chính của vấn đề hoặc cho các nguyên do khác.

Sử dụng JDB Logger

J2SE Logging Framework được thiết kế để cung cấp các dịch vụ log cho bất kỳ class nào đang chạy trong JVM. Vì thế, chúng ta có thể sử dụng Framework này để log bất kỳ thông tin nào.

Chúng ta viết lại ví dụ trên, sử dụng JDK logger API:

```
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@page import="java.util.logging.Logger" %>

<html>
<head><title>Logger.info</title></head>
```

```

<body>

<% Logger logger=Logger.getLogger(this.getClass().getName());%
>

<c:forEach var="counter" begin="1" end="10" step="1" >
    <c:set var="myCount" value="-5" />
    <c:out value=""/></br>
    <% String message = "counter="
                + pageContext.findAttribute("counter")
                + " myCount="
                + pageContext.findAttribute("myCount");
        logger.info( message );

    %>
</c:forEach>
</body>
</html>

```

Code này sẽ cho kết quả giống như trên tại trình duyệt và trong stdout.log, nhưng bạn sẽ có thông tin bổ sung trong stdout.log. Tại đây, chúng ta đang sử dụng phương thức info của Logger, bởi vì chúng ta đang log các thông báo chỉ cho mục đích thông tin. Dưới đây là snapshot (ảnh chụp màn hình) của stdout.log file:

```

24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=1 myCount=-4
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=2 myCount=-3
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=3 myCount=-2
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=4 myCount=-1
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=5 myCount=0
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=6 myCount=1
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=7 myCount=2
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=8 myCount=3
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=9 myCount=4
24-Sep-2010 23:31:31 org.apache.jsp.main_jsp _jspService
INFO: counter=10 myCount=5

```

Các message có thể được gửi tại các level khác nhau bởi sử dụng các hàm như: **severe()**, **warning()**, **info()**, **config()**, **fine()**, **finer()**, và **finest()**. Tại đây, phương thức **finest()** có thể được sử dụng để log thông tin **finest** và phương thức **severe()** để log thông tin **severe**.

Bạn có thể sử dụng **Log4J Framework** để log các thông báo trong các file khác nhau dựa trên các mức độ và tính quan trọng.

Công cụ Debug trong JSP

NetBeans là một môi trường lập trình miễn phí và mã nguồn mở được tích hợp với Java. Môi trường, mà hỗ trợ phát triển các Java application và Web application độc lập, hỗ trợ JSP và Servlet và cũng bao gồm các JSP Debugger.

NetBeans hỗ trợ các tính năng debug cơ bản sau:

- Breakpoints
- Stepping through code
- Watchpoints

Bạn có thể tham khảo NetBeans Documentation để hiểu chi tiết các tính năng debug trên.

Sử dụng JDB Debugger

Bạn có thể debug các chương trình JSP và Servlet với các lệnh jdb tương tự như khi bạn sử dụng để debug một applet hoặc một application.

Để debug một JSP hoặc Servlet, chúng ta có thể debug `sun.servlet.http.HttpServer`, sau đó quan sát khi `HttpServer` thực thi các Servlet này trong phản hồi tới các HTTP Request chúng ta tạo từ một trình duyệt. Nó tương tự như cách các applet được debug. Điểm khác nhau là với applet, chương trình thực sự được debug là `sun.applet.AppletViewer`.

Hầu hết debugger ẩn chi tiết này bởi việc tự động biết cách để debug các applet. Tới khi chúng thực hiện điều tương tự với các Servlet, bạn phải giúp debugger của mình bởi thực hiện các điều sau:

- Thiết lập Classpath của debugger của bạn để mà nó có thể tìm `sun.servlet.http.Http-Server` và các class liên kết.
- Thiết lập Classpath của debugger để mà nó cũng có thể tìm các JSP của bạn và hỗ trợ các class, đặc trưng là `ROOT\WEB-INF\classes`.

Khi đã thiết lập classpath thích hợp, bắt đầu việc debug `sun.servlet.http.HttpServer`. Bạn có thể thiết lập các Breakpoint trong bất kỳ JSP nào bạn quan tâm, sau đó sử dụng một trình duyệt web để tạo một yêu cầu tới `HttpServer` cho JSP đã cho (`http://localhost:8080/JSPToDebug`). Bạn nên quan sát việc dừng sự thực thi tại các Breakpoint.

Sử dụng Comment trong JSP

Các Comment trong code của bạn có thể giúp tiến trình debug theo nhiều cách. Các comment có thể được sử dụng theo nhiều cách trong tiến trình debug.

JSP sử dụng Java comment và các comment đơn dòng (`// ...`) và đa dòng (`/* ... */`) có thể được sử dụng để tạm thời gỡ bỏ các phần của Java code. Nếu bug biến mất, quan sát cẩn thận code mà bạn đã comment và tìm ra vấn đề.

Các trường Header trên Client và Server

Đôi khi, khi một JSP không vận hành như mong đợi, nó là hữu ích khi nhìn vào HTTP Request và HTTP Response thô. Nếu bạn chưa làm quen với cấu trúc của HTTP, bạn có thể đọc Request và Response và xem chính xác những gì đang diễn ra với các Header này.

Lời khuyên tốt cho Debug trong JSP

Sau đây là một số lời khuyên cho việc debug chương trình JSP:

Yêu cầu các trình duyệt hiển thị nội dung thô của trang nó đang hiển thị. Điều này giúp nhận diện các vấn đề về định dạng. Nó thường là một option ở dưới View menu.

Đảm bảo rằng trình duyệt không cache kết quả của yêu cầu trước đó bằng việc bắt buộc Full Reload trang. Với Netscape Navigator, sử dụng Shift-Reload; với Internet Explorer, sử dụng Shift-Refresh.

Câu hỏi ôn tập.ⁱ

4.18. Bảo mật trong JSP

JavaServer Pages và Servlet cung cấp một số kỹ thuật có sẵn tới lập trình viên để bảo mật các ứng dụng. Nguồn được bảo vệ bởi việc nhận diện chúng trong Application Deployment Descriptor và gán một role cho chúng.

Một số mức độ xác nhận (Authentication) là có sẵn, từ Basic Authentication (xác nhận cơ bản) bởi sử dụng Identifier và Password cho tới Sophisticated Authentication (xác nhận tinh vi hơn) bởi sử dụng Certificate.

Role dựa trên Authentication

Kỹ thuật Authentication trong Servlet Specification sử dụng một kỹ thuật gọi là Role-Based Security. Ý tưởng này thay cho việc hạn chế các nguồn (restricting resources) tại level người sử dụng, bạn tạo các role và hạn chế nguồn bởi role đó.

Bạn có thể định nghĩa các role khác nhau trong tomcat-users.xml file, mà được đặt tại cuối của Home directory trong Tomcat trong conf. Một ví dụ của file này là:

```
<?xml version='1.0' encoding='utf-8'?>
<tomcat-users>
<role rolename="tomcat"/>
<role rolename="role1"/>
```

```

<role rolename="manager"/>
<role rolename="admin"/>
<user username="tomcat" password="tomcat" roles="tomcat"/>
<user username="role1" password="tomcat" roles="role1"/>
<user username="both" password="tomcat" roles="tomcat,role1"/>
<user username="admin" password="secret" roles="admin,manager"
/>

</tomcat-users>

```

File này định nghĩa một ánh xạ đơn giản giữa user name, password và role. Chú ý rằng, một người sử dụng đã cho có nhiều role, ví dụ, user name="both" là trong "tomcat" role và "role1" role.

Khi bạn đã nhận diện và định nghĩa các role khác nhau, một sự hạn chế Role-Based Security có thể được đặt trên các nguồn Web application khác nhau bởi sử dụng phần tử **<security-constraint>** trong web.xml file có sẵn trong thư mục WEB-INF.

Ví dụ sau là một entry mẫu trong web.xml:

```

<web-app>
...
    <security-constraint>
        <web-resource-collection>
            <web-resource-name>
                SecuredBookSite
            </web-resource-name>
            <url-pattern>/secured/*</url-pattern>
            <http-method>GET</http-method>
            <http-method>POST</http-method>
        </web-resource-collection>
        <auth-constraint>
            <description>
                Let only managers use this app
            </description>
            <role-name>manager</role-name>
        </auth-constraint>
    </security-constraint>
    <security-role>

```

```

        <role-name>manager</role-name>
    </security-role>
    <login-config>
        <auth-method>BASIC</auth-method>
    </login-config>
    ...
</web-app>

```

Các entry trên nghĩa là:

- Bất kỳ HTTP GET hoặc POST yêu cầu tới một URL đã kết nối bởi /secured/* sẽ là chủ đề tới Security Restriction.
- Một người với manager role được cung cấp sự truy cập tới các nguồn được bảo mật.
- Cuối cùng, phần tử login-config được sử dụng để miêu tả Basic Authentication.

Bây giờ, nếu bạn thử trình duyệt tới bất kỳ URL nào bao gồm thư mục /security, nó sẽ hiển thị một hộp thoại yêu cầu user name và password. Nếu bạn cung cấp một user là "admin" và password là "secrecr", thì chỉ bạn sẽ có quyền truy cập trên URL đã được kết nối bởi /secured/*, bởi vì ở trên chúng ta đã định nghĩa user admin với manager role, người mà được phép truy cập nguồn này.

Form dựa trên Authentication

Khi bạn sử dụng phương thức FORM authentication, bạn phải cung cấp một login form để nhắc người dùng về một username và một password. Sau đây là code đơn giản login.jsp để tạo một form cho mục đích trên:

```

<html>
<body bgcolor="#ffffff">
    <form method="POST" action="j_security_check">
        <table border="0">
            <tr>
                <td>Login</td>
                <td><input type="text" name="j_username"></td>
            </tr>
            <tr>
                <td>Password</td>
                <td><input type="password" name="j_password"></td>
            </tr>
        </table>
    </form>

```

```

        <input type="submit" value="Login!">
    </center>
</form>
</body>
</html>

```

Tại đây bạn phải đảm bảo rằng, login form phải chứa các phần tử form với tên `j_username` và `j_password`. Action trong thẻ `<form>` phải là `j_security_check`. Phương thức POST phải được sử dụng như là phương thức của form. Cùng lúc đó, bạn phải sửa đổi thẻ `<login-config>` để xác định auth-method là FORM:

```

<web-app>
...
    <security-constraint>
        <web-resource-collection>
            <web-resource-name>
                SecuredBookSite
            </web-resource-name>
            <url-pattern>/secured/*</url-pattern>
            <http-method>GET</http-method>
            <http-method>POST</http-method>
        </web-resource-collection>
        <auth-constraint>
            <description>
                Let only managers use this app
            </description>
            <role-name>manager</role-name>
        </auth-constraint>
    </security-constraint>
    <security-role>
        <role-name>manager</role-name>
    </security-role>
    <login-config>
        <auth-method>FORM</auth-method>
        <form-login-config>

```

```

        <form-login-page>/login.jsp</form-login-page>
        <form-error-page>/error.jsp</form-error-page>
    </form-login-config>
</login-config>

...
</web-app>

```

Lúc này, khi bạn thử truy cập bất kỳ nguồn nào với URL là /security/*, nó sẽ hiển thị form trên, yêu cầu cho user id và password. Khi container thấy "j_security_check" action, nó sử dụng một số kỹ thuật nội vi để xác nhận khách truy cập.

Nếu việc login là thành công và khách truy cập được cho quyền truy cập tới các nguồn bảo mật, thì khi đó container sử dụng một session-id để nhận diện login session cho khách truy cập từ thời điểm đó. Container duy trì login session với một cookie chứa session-id đó. Server gửi cookie trở lại tới Client, và miễn là khách truy cập đưa ra cookie này với các request liên tục, thì container sẽ biết khách truy cập đó là ai.

Nếu login thất bại, thì Server gửi trả lại trang đó một thiết lập form-error-page

Tại đây j_security_check là action mà các ứng dụng sử dụng form dựa trên login phải xác định login form đó. Trong cùng form đó bạn cũng nên có một điều khiển Text input, gọi là j_username và một password gọi là j_password. Khi bạn thấy điều này, nghĩa là thông tin được chứa trong form đó sẽ được đệ trình tới Server, mà sẽ kiểm tra name và password. Đó là cách thực hiện với một Server cụ thể.

Bạn kiểm tra Standard Realm Implementations để hiểu cách j_security_check làm việc cho Tomcat container.

Bảo mật tự động trong một Servlet/JSP

Đối tượng HttpServletRequest cung cấp các phương thức có thể được sử dụng để đặt thông tin bảo mật tại runtime.

STT	Phương thức and Description
1	String getAuthType() Phương thức getAuthType() trả về một đối tượng String mà biểu diễn tên của Authentication Scheme được sử dụng để bảo vệ Servlet
2	boolean isUserInRole(java.lang.String role) Phương thức isUserInRole() trả về true nếu người dùng là ở một role đã cho, trả về false nếu họ là không

3	String getProtocol() Phương thức getProtocol() trả về một đối tượng String biểu diễn giao thức được gửi request. Giá trị này có thể được kiểm tra để quyết định nếu một giao thức an toàn được sử dụng
4	boolean isSecure() Phương thức isSecure() method trả về giá trị true biểu diễn nếu request được tạo bởi sử dụng HTTPS. Giá trị true nghĩa là nó và kết nối là an toàn. Giá trị false nghĩa là yêu cầu là không an toàn
5	Principle getUserPrinciple() Phương thức getUserPrinciple() trả về một đối tượng java.security.Principle chứa tên người dùng đã được xác nhận hiện tại

Ví dụ, một JavaServer Page mà liên kết với các trang có role là manager, bạn có các dòng code sau:

```
<% if (request.isUserInRole("manager")) { %>
<a href="managers/mgrreport.jsp">Manager Report</a>
<a href="managers/personnel.jsp">Personnel Records</a>
<% } %>
```

Bằng việc kiểm tra role của người sử dụng trong một JSP hoặc Servlet, bạn có thể tùy chỉnh Webpage để hiển thị các item mà chỉ người đó được truy cập. Nếu bạn cần tên của người dùng khi họ nhập trong Authentication Form, bạn có thể gọi phương thức getRemoteUser trong đối tượng request.

Câu hỏi ôn tập.

1. Mô tả cách xử lý ngoại lệ trong JSP và cách sử dụng thẻ error-page trong tệp web.xml.
2. làm thế nào để hiển thị thông báo lỗi cụ thể cho người dùng trong trang JSP?
3. Giải thích cách sử dụng đối tượng exception để hiển thị thông báo lỗi chi tiết cho người dùng.
4. làm thế nào để bảo vệ dữ liệu được truyền trong URL Trong JSP?.
5. Mô tả cách sử dụng thẻ <c:out> và hàm fn:escapeXml để bảo vệ dữ liệu truyền qua URL.
6. Hướng dẫn cách sử dụng thẻ <c:if> và <c:when> để kiểm tra và đảm bảo an toàn dữ liệu nhập từ người dùng.

7. Mô tả cách sử dụng PreparedStatement và Parameterized Queries để bảo vệ ứng dụng JSP khỏi SQL Injection.

BÀI TẬP THỰC HÀNH

Bài 1.

Tạo một trang đăng ký tài khoản người dùng với các yêu cầu sau:

- Servlet: RegisterServlet
- Route: /dang-ky-tai-khoan.jsp
- Trường dữ liệu cần quản lý : username, họ tên, mật khẩu, tuổi, địa chỉ.

Khi người dùng click Register trong trang đăng ký, Lưu thông tin vào CSDL thì nhảy sang trang login yêu cầu như sau:

- Servlet : LoginServlet
- Route: /dang-nhap.jsp
- Form gồm các trường sau: username, mật khẩu
- Khi người dùng click vào Login thì thực hiện kiểm tra điều kiện sau. Kiểm tra thông tin đăng nhập, Đúng thì vào trang welcome

Nếu khớp: Chuyển sang trang welcome

Yêu cầu:

- Servlet: WelcomeServlet
- Route: xin-chao.jsp
- Trang JSP: welcome.jsp, Hiển thị thông tin người dùng ra và danh sách người dùng

Bài 2.

Xây dựng trang thông-tin.jsp (Chỉ sử dụng JSP) để tìm kiếm thông tin cá nhân của một người dùng

- Thông tin cá nhân gồm các trường: CMTND, họ tên, địa chỉ, sdt, email, ngày sinh.
- Mục tìm kiếm : gồm form chứa một input search (nhập CMTND hoặc email hoặc sdt)
- Mục hiển thị thông tin tìm kiếm, hiển thị vào một bảng
- Thực hiện : Tách trang thành các phần : search.jsp và data.jsp và hiển thị trong trang thông-tin.jsp.

Bài 3.

1. Xây dựng các trang JSP để quản lý tin tức được mô tả như sau:

-Thiết kế CSDL đặt tên là QuanLyTinTuc gồm một bảng tintuc với các trường dữ liệu sau:id(trường khóa chính tự tăng), tiêu đề, thumbnail(lưu trữ url hình ảnh của tin tức), ngày tạo.

-Thiết kế form nhập thông tin tin tức

-Hiện thị danh sách tin tức

-Xem chi tiết từng tin tức

Bài 4.

Sử dụng Expression Language/JSTL thực hiện hiển thị giao diện sau:

-Tạo lớp JavaBean Student gồm các thuộc tính sau : no, fullname, gender, email, address.

-Tạo một servlet đặt tên là StudentServlet vs URL là danh-sach-sinh-vien.

-Tạo mảng dữ liệu sinh viên trong StudentServlet , thực hiện gửi dữ liệu sang trang studentlist.jsp để hiển thị như hình sau:

No	Fullname	Gender	Email	Address
01	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh
02	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh
03	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh
04	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh
05	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh
06	Tran Van Diep	Nam	tranvandiep.it@gmail.com	Nam Dinh

B

- Danh mục sản phẩm gồm các trường : id, tên danh mục

- Sản phẩm gồm các trường : id, id danh mục, tiêu đề, thumbnail, nội dung

- Đơn hàng gồm : id, ngày tạo, tên khách hàng, sdt khách hàng, địa chỉ, tổng tiền

- Chi tiết đơn hàng : id, id đơn hàng, id sản phẩm, số lượng, tổng tiền, giá

Yêu cầu:

- Tạo các chức năng thêm/sửa/xoá sản phẩm

- Thiết kế luồng mua hàng.

- Quản lý đơn hàng.

Tài liệu tham khảo

Bài giảng nội bộ chia thành từng bài.

